

Analisi delle performance di 3 modelli di machine learning su un dataset progressivamente sporcato

Andrea Spagnolo 879254

Introduzione

Introduzione	1
Processo di degradazione dataset	2
Valori nulli	2
Implementazione	3
Analisi risultati	4
valori duplicati	13
implementazione	13
Analisi risultati	13
outliers	18
Implementazione	19
Analisi risultati	21
rumore su feature target	26
implementazione	26
Analisi risultati	27
Conclusioni	31
Codice Sorgente	32

L'obiettivo di questo progetto è stato analizzare l'impatto di diverse tipologie di errori sulla qualità di un dataset. Ho voluto studiare in che modo la degradazione progressiva dei dati influenzi le performance di tre modelli di machine learning distinti: **Decision Tree**, **Support Vector Machine (SVM)** e **Neural Network**.

Per ogni fase di degradazione, ho addestrato e valutato ciascuno dei tre modelli sia nella loro versione **naïve** che in quella **ottimizzata**. Questa doppia valutazione mi ha permesso di capire meglio se e come l'ottimizzazione dei parametri possa mitigare gli effetti negativi degli errori presenti nel dataset.

La mia analisi si è concentrata sul monitoraggio delle metriche di **accuracy**, **precision**, **recall**, **F1-score** e **delle matrici di confusione**, man mano che la qualità del dataset diminuiva. L'obiettivo principale era studiare l'evoluzione delle performance dei modelli al peggiorare della qualità del dataset. Quello che mi aspetto è di osservare una correlazione

diretta tra l'aumento degli errori e un calo delle performance, ma con variazioni significative tra i diversi modelli e le loro versioni.

Processo di degradazione dataset

Per il processo di degradazione della qualità del dataset sono stati usati diversi approcci:

1. valori nulli
2. valori duplicati
3. outliers
4. rumore sulle variabili binarie
5. rumore sulla variabile target

Per ognuno di questi approcci è stata utilizzata la seguente pipeline di lavoro:

1. Richiamo dal dataset preprocessato
2. Divisione in feature binarie/categoriche e continue
3. Calcolo mutua informazione feature su dataset pulito
4. Applicazione procedura di degradazione dataset e calcolo mutua informazione delle feature per ogni step di degradazione del dataset
5. Visualizzazione dataset degradato
6. Addestramento modelli su ogni step di degradazione del dataset e calcolo delle relative metriche (accuracy, recall ...) e matrici di confusione
7. Visualizzazione andamento metriche al crescere del livello di degradazione del dataset
8. Visualizzazione andamento matrici di confusione al crescere del livello di degradazione del dataset
9. Analisi sull'evoluzione dell'importanza delle feature nel decision tree al crescere del livello di degradazione delle feature
10. Analisi sull'evoluzione della mutua informazione al crescere dello sporcamento del dataset

Valori nulli

La presenza di valori mancanti nei dataset è un problema estremamente comune e pervasivo nel mondo reale. Le ragioni sono molteplici: guasti dei sensori, errori di inserimento dati, sondaggi a cui non si risponde completamente ...

Quindi essendo un problema molto diffuso analizzare come i valori nulli vadano ad impattare sulle performance risulta molto utile in quanto ci permette di valutare quali sono i **modelli più robusti** a questo tipo di problema e quali sono i **metodi di imputazione più efficaci** da utilizzare. Inoltre, l'approccio di degradazione a step ci consente di identificare il **punto di rottura** del modello, ovvero la percentuale di dati mancanti oltre la quale le sue performance iniziano a crollare drasticamente. Conoscere questo valore è fondamentale in quanto ci fornisce una soglia di accettabilità per la qualità dei dati, aiutandoci a scartare i dataset non sufficientemente "solidi" per un'analisi significativa.

Implementazione

Per implementare questo processo di degradazione abbiamo utilizzato la pipeline precedentemente descritta, ma con dei passaggi aggiuntivi dovuti all'analisi del migliore metodo di imputazione per ogni modello.

Di seguito la funzione utilizzata per inserire progressivamente i valori mancanti nel dataset

```
def introduce_missing_values(df: pd.DataFrame, columns: list, percentage: float) -> pd.DataFrame:

    df_degraded = df.copy()
    for col in columns:
        non_missing_indices = df_degraded[col].dropna().index
        n_to_make_missing = int(len(non_missing_indices) * percentage)

        missing_indices = np.random.choice(
            non_missing_indices,
            size=n_to_make_missing,
            replace=False
        )
        df_degraded.loc[missing_indices, col] = np.nan

    return df_degraded
```

Questa funzione è stata applicata a tutto il dataset esclusa la colonna target

```
features_to_degrade = df.columns.drop("HeartDisease")
```

La funzione è poi stata lanciata per ogni step di degradazione, ovvero da 0 a 90 con intervalli di 10.

```
for p in missing_percentages:
    print(f"Generando dataset con {int(p*100)}% di valori mancanti...")
    degraded_datasets[p] = introduce_missing_values(
        df=df,
        columns=features_to_degrade,
        percentage=p
    )
```

Una volta ottenuti i vari dataset degradati in maniera progressiva sono stati addestrati i vari modelli (decision tree, svm e neural networks) sia nella loro versione naive che ottimizzata. Inoltre per ognuno di questi modelli sono stati provati diversi metodi di imputazione per studiare il più efficace. In particolare abbiamo sostituito i dati mancanti utilizzando la media, la mediana, la moda e come metodologie più sofisticate abbiamo provato knn e il metodo iterativo.

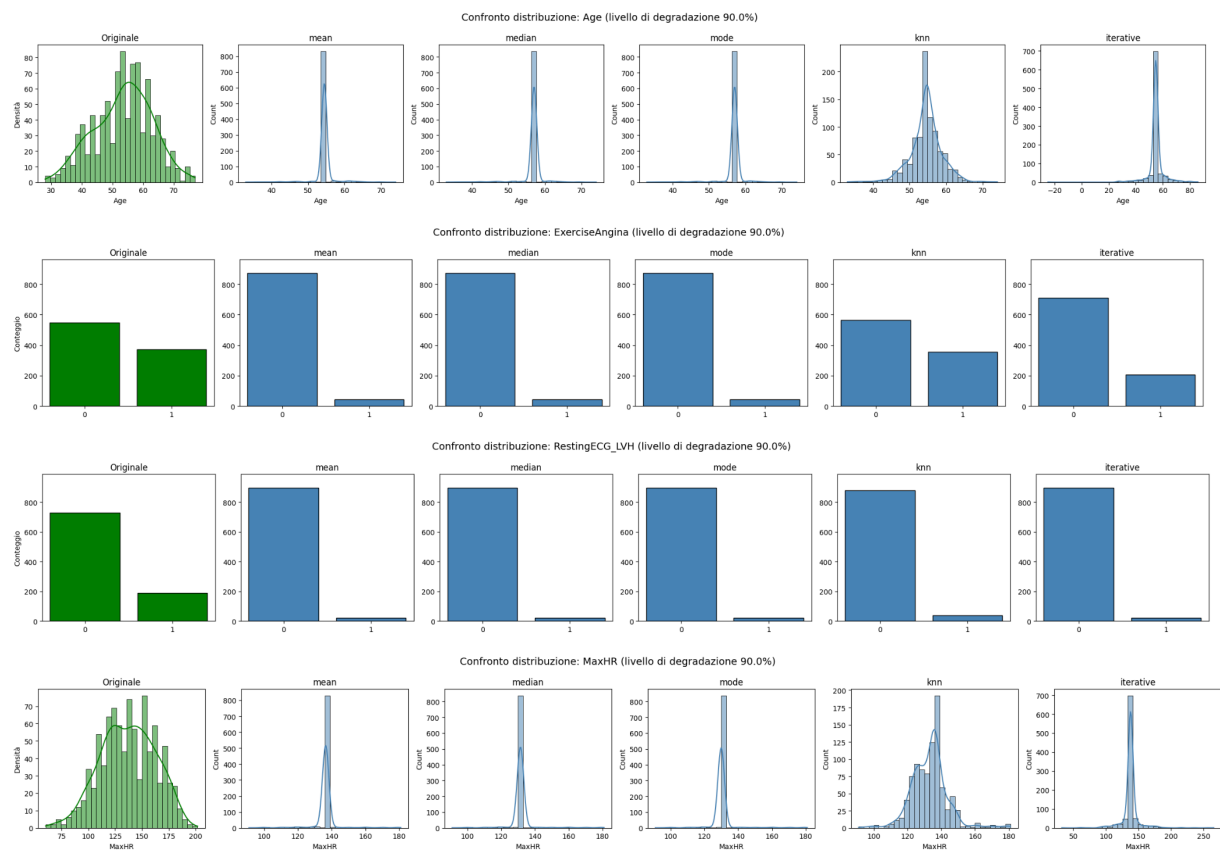
```
imputation_strategies = {
    'mean': SimpleImputer(strategy='mean'),
    'median': SimpleImputer(strategy='median'),
    'mode': SimpleImputer(strategy='most_frequent'),
    'knn': KNNImputer(n_neighbors=5),
    'iterative': IterativeImputer(random_state=42)
}
```

Una volta completata l'imputazione dei valori mancanti, mi sono assicurato della validità dei dati prodotti. Ho riscontrato che, per le feature binarie, il metodo di imputazione della media generava valori non binari (ad esempio, 0.2, 0.5, 0.8), che non erano coerenti con il dominio del dato (0 o 1). Per correggere questo problema e garantire l'integrità del dataset, ho applicato una soglia di discretizzazione. Il codice riportato di seguito rende i valori binari, arrotondando a 1 quelli superiori a 0.5 e a 0 quelli inferiori o uguali a 0.5

```
for col in binary_features:
    if col in X_train_imp_df.columns:
        X_train_imp_df[col] = (X_train_imp_df[col] > 0.5).astype(int)
        X_test_imp_df[col] = (X_test_imp_df[col] > 0.5).astype(int)
```

Analisi risultati

Come prima cosa ho ritenuto utile andare a rappresentare la distribuzione ottenuta alla fine del procedimento di degradazione del dataset, per osservare anche quali tra i metodi di imputazione usati andavano a riprodurre meglio la distribuzione originale.

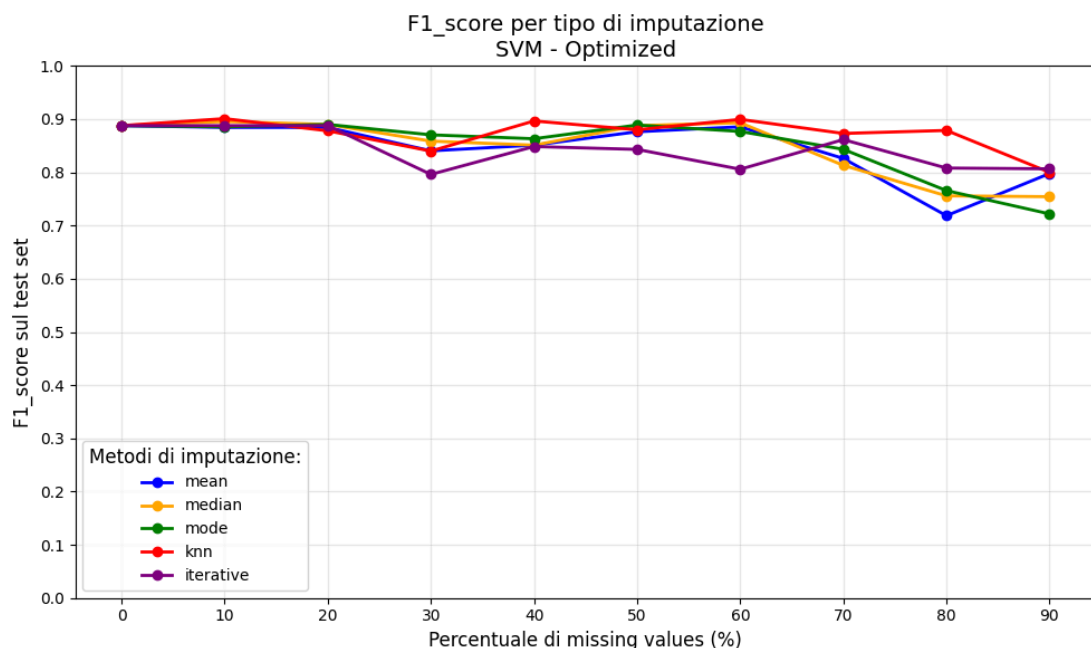
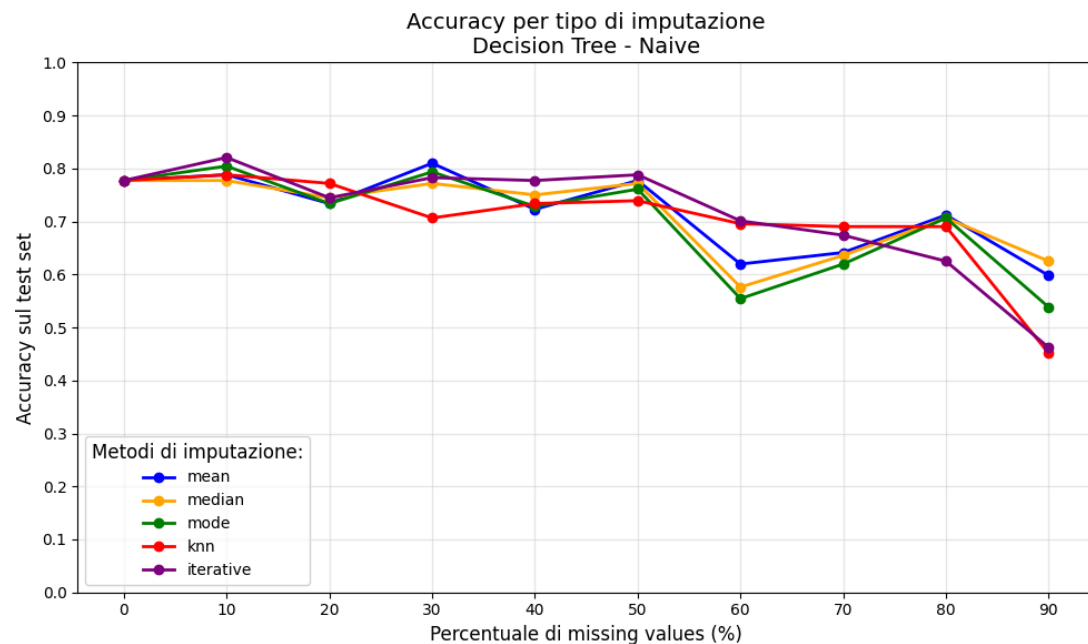


Per osservare i grafici delle distribuzioni di tutte le feature guardare nel file [missing_values](#) su [github](#).

I grafici mostrano delle distribuzioni fortemente compresse, sintomo che la nostra funzione di degradazione ha lavorato correttamente. Le distribuzioni più compromesse risultano essere quelle continue, in cui solo il metodo di imputazione knn sembra restituire una distribuzione

minimamente paragonabile. Anche le feature binarie risultano compromesse con metodi di imputazione “leggeri” come media, moda e mediana, mentre con knn e il metodo iterativo si mantengono più simili alla distribuzione originale. Da questa prima analisi risulta quindi che i metodi knn e il metodo iterativo, seppur molto più onerosi a livello computazionale, sembrano essere l’unica scelta praticabile se si ha una percentuale di valori mancanti molto alta.

Per analizzare i risultati ho creato dei grafici che mostrano l’andamento di ogni singola metrica per ogni modello utilizzato in modo tale da analizzare il comportamento di quella singola metrica e inoltre studiare quale fosse il migliore metodo di imputazione. Di seguito riporto qualche grafico di esempio:



Questa funzione andava a produrre 4 grafici (uno per metrica) per ogni singolo modello e per ogni singolo approccio, per un totale di 24 grafici il che rende poco immediata la comprensione dei risultati. A fronte di ciò ho prodotto dei nuovi grafici in cui andavo a selezionare il metodo di

imputazione che mediamente aveva registrato le performance migliori in questo modo riuscivo a mostrare l'andamento di tutte e 4 le metriche in un singolo grafico facendo scendere il numero di grafici prodotti da 24 a soli 6 grafici

```
for model in models:
    for approach in approaches:
        approach_df = metrics_df[
            (metrics_df["approach"] == approach) &
            (metrics_df["model"] == model)
        ]

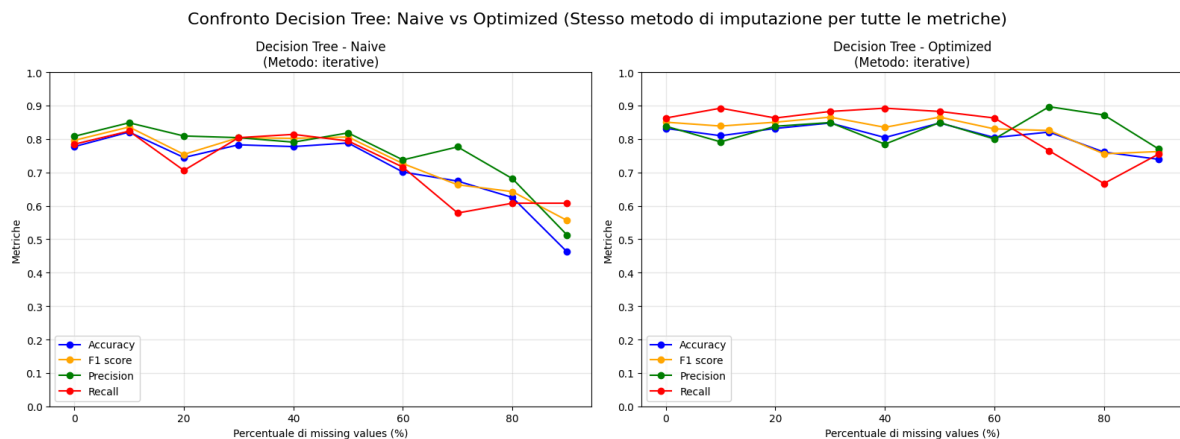
        # Calcola la media di TUTTE le metriche per ogni metodo di imputazione
        imputation_methods = approach_df['imputation'].unique()

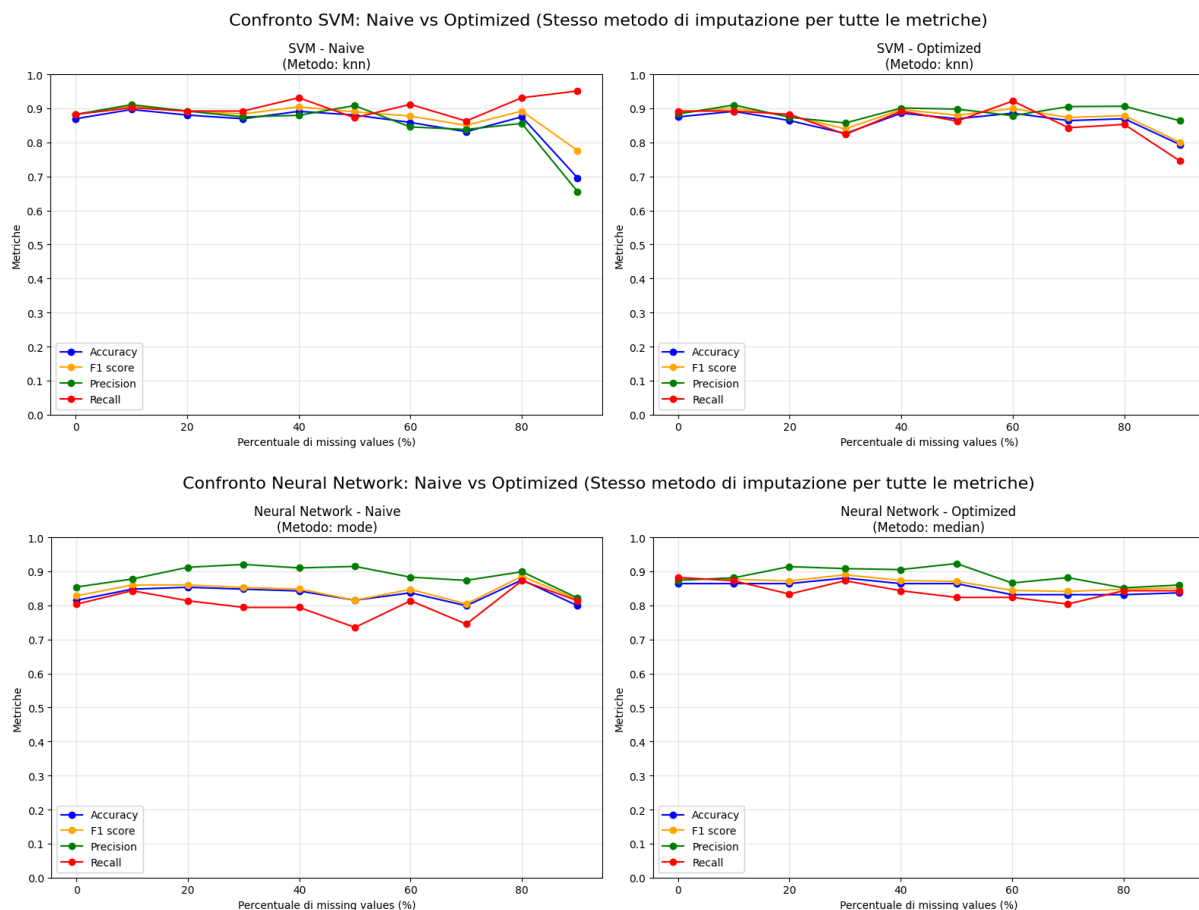
        for imp_method in imputation_methods:
            imp_df = approach_df[approach_df['imputation'] == imp_method]
            avg_score = imp_df[metric_names].mean().mean()

            results.append({
                'Model': model,
                'Approach': approach,
                'Imputation': imp_method,
                'Avg_Score': avg_score
            })

results_df = pd.DataFrame(results)

# Trova il migliore per ogni combinazione modello-approccio
best_results = results_df.loc[results_df.groupby(['Model', 'Approach'])['Avg_Score'].idxmax()]
```



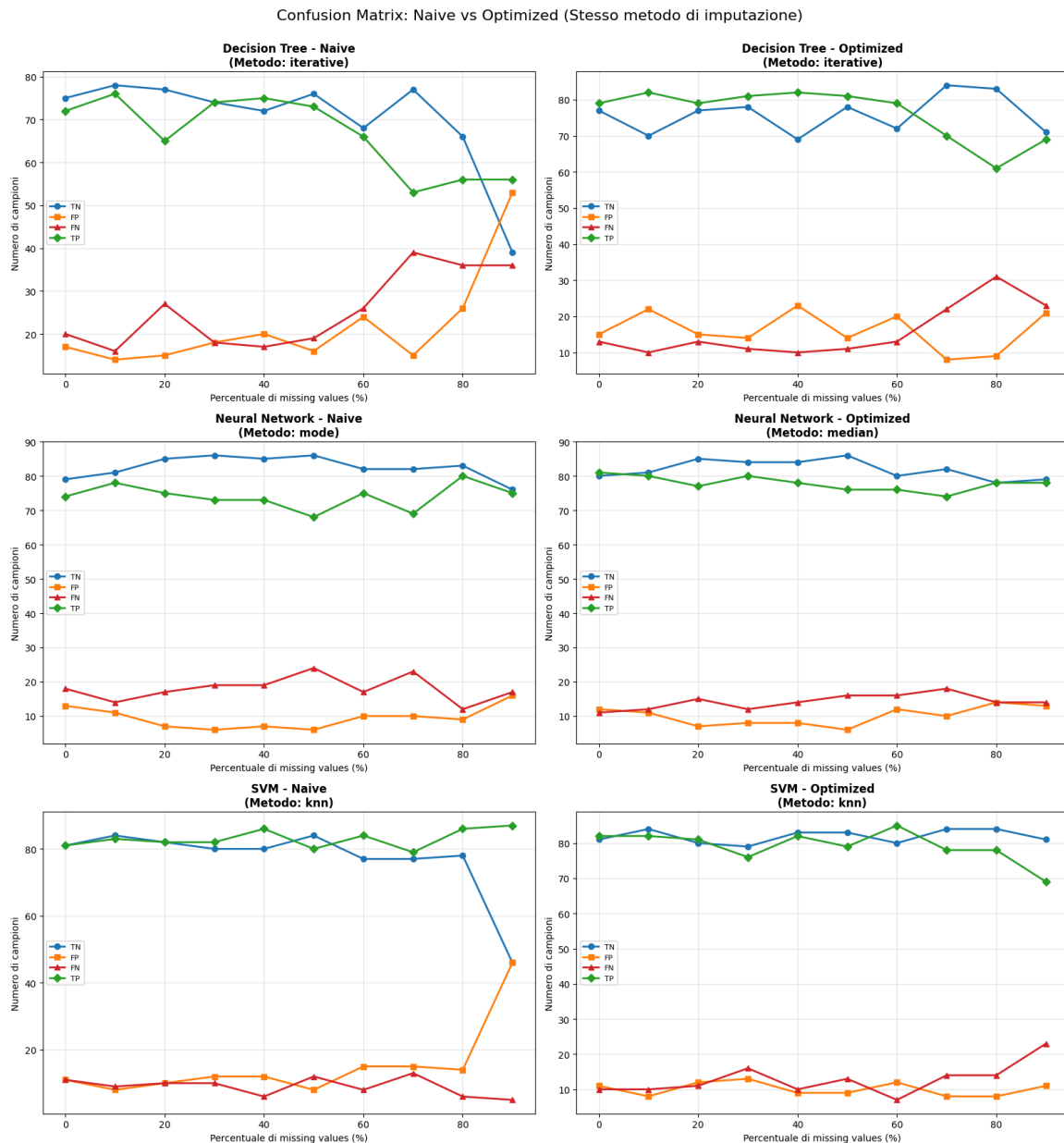


Osservando i grafici risulta evidente che sia la **rete neurale** sia **SVM** si distinguono per una maggiore stabilità e solidità nelle prestazioni, riuscendo a mantenere valori elevati nelle principali metriche anche all'aumentare della percentuale di dati mancanti. Entrambi mantengono valori di accuracy, precision e f1-score molto elevati e stabili fino a circa il 70-80% di missing values, mostrando solo nelle ultime due fasce una flessione visibile, ma comunque inferiore rispetto a quella rilevata nel decision tree, suggerendo come questi due modelli siano generalmente più resistenti alla perdita di informazione se supportati da un buon metodo di imputazione standard. Al contrario, il **decision tree** conferma la sua maggiore sensibilità ai dati mancanti, soprattutto nell'approccio naive: superata la soglia del 60% di missing, tutte le metriche subiscono un calo marcato e, oltre l'80%

Un'osservazione importante è che i modelli ottimizzati, come prevedibile, hanno mostrato performance nettamente migliori rispetto ai loro corrispettivi naive, con andamenti molto più stabile delle metriche. Questo trend, che vedremo si ripeterà anche per tutte le altre analisi, può essere spiegato dal fatto che modelli di machine learning ottimizzati hanno una maggiore capacità di gestione del rumore e delle imperfezioni nei dati, a differenza dei modelli naive, che spesso sono sbilanciati e vulnerabili a piccole variazioni del dataset, i modelli ottimizzati trovano il giusto equilibrio tra bias e varianza. Questo significa che sono meno inclini a overfitting e a diventare ipersensibili ai singoli dati corrotti. In pratica, mentre un modello naive costruisce i suoi confini decisionali senza alcuna difesa contro il rumore, un modello ottimizzato utilizza i suoi iperparametri, come la profondità massima di un albero decisionale o i fattori di regolarizzazione, come dei veri e propri "ammortizzatori". Questi meccanismi

impediscono che anche pochi dati rumorosi alterino drasticamente il comportamento del modello, garantendo che i suoi risultati rimangano coerenti e affidabili anche quando la qualità del dataset degrada.

Oltre ad analizzare le performance delle metriche ho studiato anche l'evoluzione dei valori delle matrici di confusione al crescere della percentuale di valori mancanti.

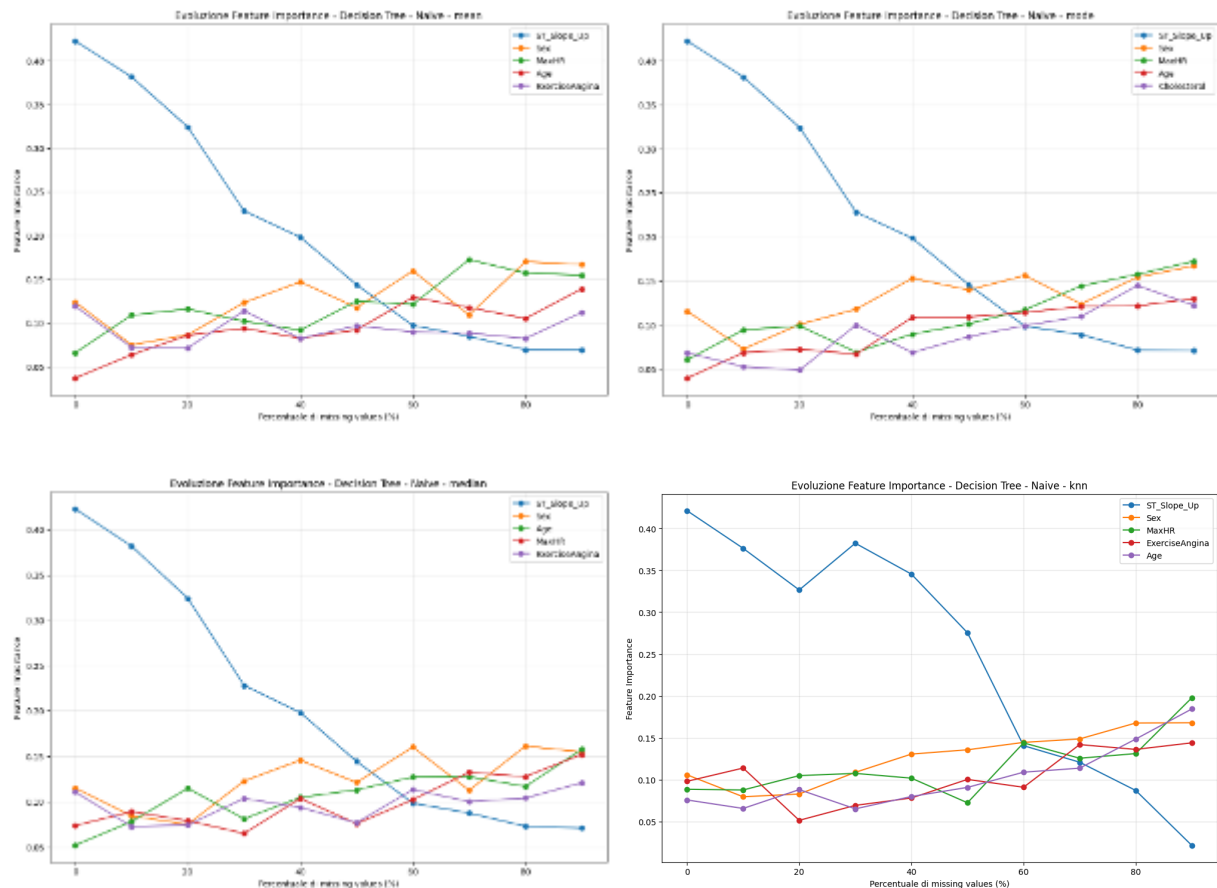


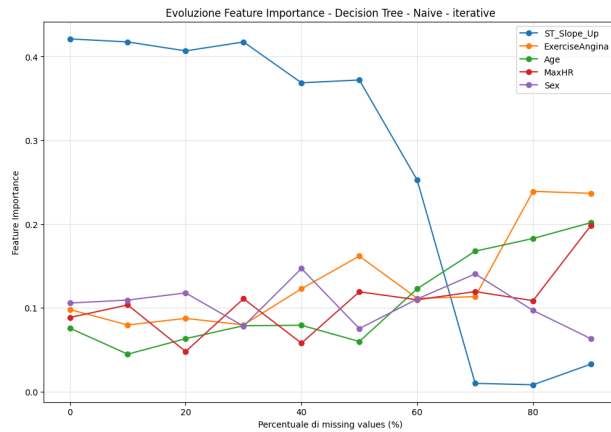
il grafico mostra l'andamento delle matrici di confusione per Decision Tree, Neural Network e SVM, sia naive sia ottimizzati, con variazione della percentuale di dati mancanti e utilizzando per ogni modello il metodo di imputazione che ha avuto le performance mediamente migliore. Si osserva che per Neural Network e SVM, sia nella versione naive che ottimizzata, il numero di veri positivi (TP) e veri negativi (TN) rimane elevato e stabile fino a circa l'80% di missing values, limitando il numero di falsi positivi (FP) e falsi negativi (FN) anche in condizioni difficili. Tuttavia, nel caso specifico dello SVM naive con il 90% di valori mancanti, si registra un calo

improvviso delle performance, con un aumento molto marcato del numero di falsi positivi (FP) e una riduzione dei veri negativi (TN). Decision Tree mostra una maggiore variabilità e fragilità: sia in versione naive che ottimizzata, la quantità di TP e TN si riduce sensibilmente oltre il 60% di missing, con aumenti evidenti di FP e soprattutto FN, segnalando un rapido declino dell'affidabilità nelle classificazioni con l'aumentare dei valori mancanti.

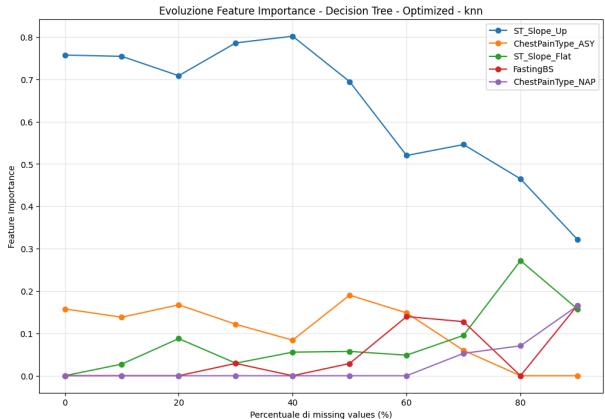
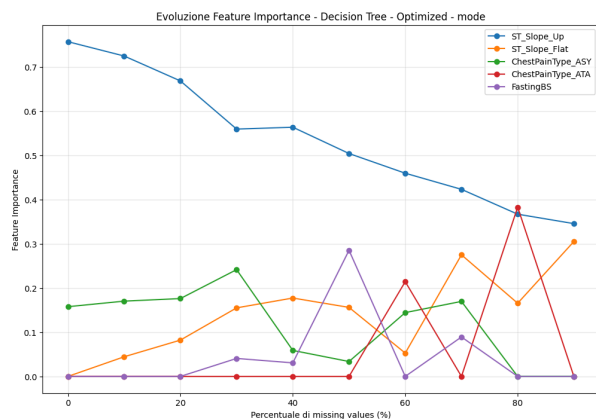
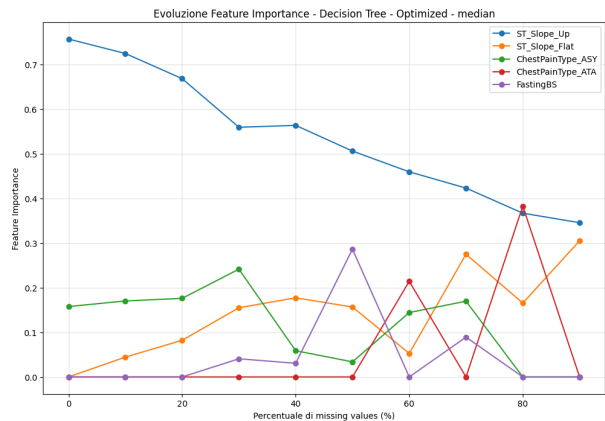
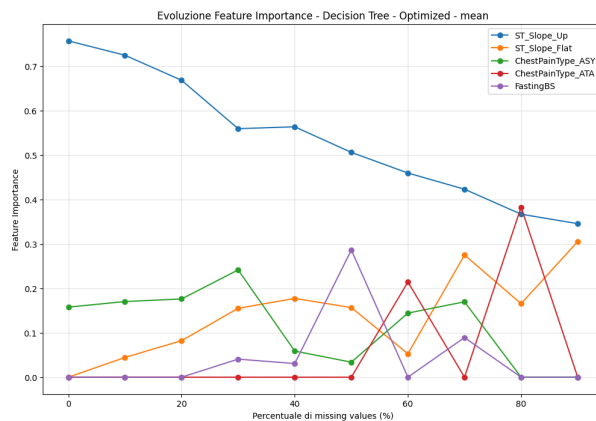
In sintesi, solo SVM (ad eccezione del caso naive al 90% missing) e Neural Network mantengono buone performance anche ad alte percentuali di dati mancanti, mentre il Decision Tree soffre maggiormente la perdita di informazione, confermando la tendenza già emersa nelle metriche generali precedentemente analizzate.

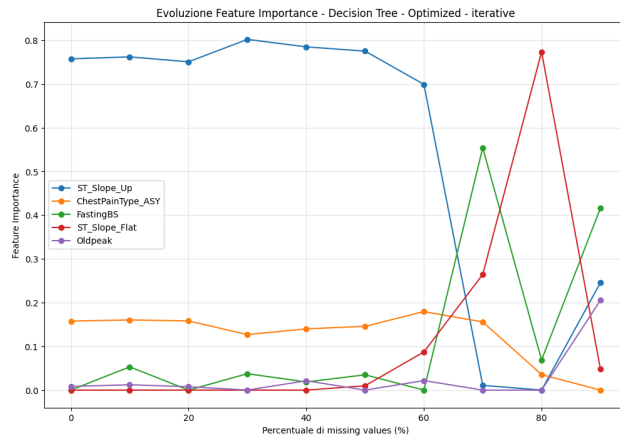
A questo punto sono andato a studiare anche l'evoluzione dell'importanza delle feature per comprendere meglio le cause dell'andamento delle metriche. Questa analisi è stato possibile svolgerla unicamente con gli alberi di decisione in quanto risulta computazionalmente molto meno oneroso. Infatti su SVM (con kernel non lineari) e reti neurali ho provato a studiare l'importanza delle metriche con shap ma per avere risultati un minimo significativi bisogna eseguirla con parametri non troppo piccoli il che comportava tempo di esecuzione di diverse ore rendendo l'analisi poco sostenibile





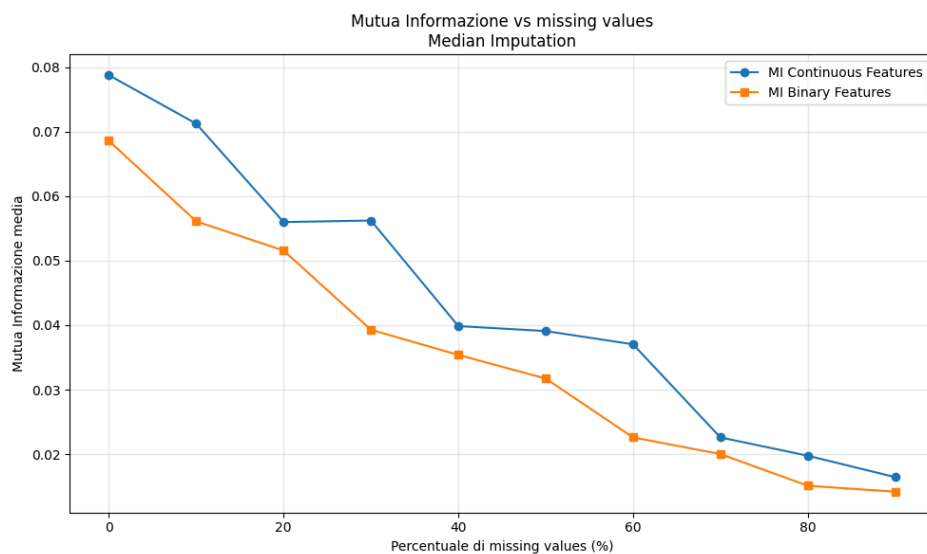
Dai grafici risulta che la feature più importante negli alberi di decisione naive era la ST_Slope_Up. Quando il modello fa crollare l'importanza di questa metrica a favore delle altre è il momento in cui si ha anche il calo di performance delle metriche più evidente sottolineando l'importanza di questa feature per questo modello. Dai grafici si può inoltre osservare come l'utilizzo di diversi metodi di imputazione non vada troppo a stravolgere l'evoluzione dell'importanza delle feature negli alberi di decisione naive. L'unica differenza degna di nota è che nei metodi di imputazione più sofisticati la feature più importante (ST_Slope_Up) rimane rilevante per più step di degradazione rispetto ai modelli di imputazione più semplici.

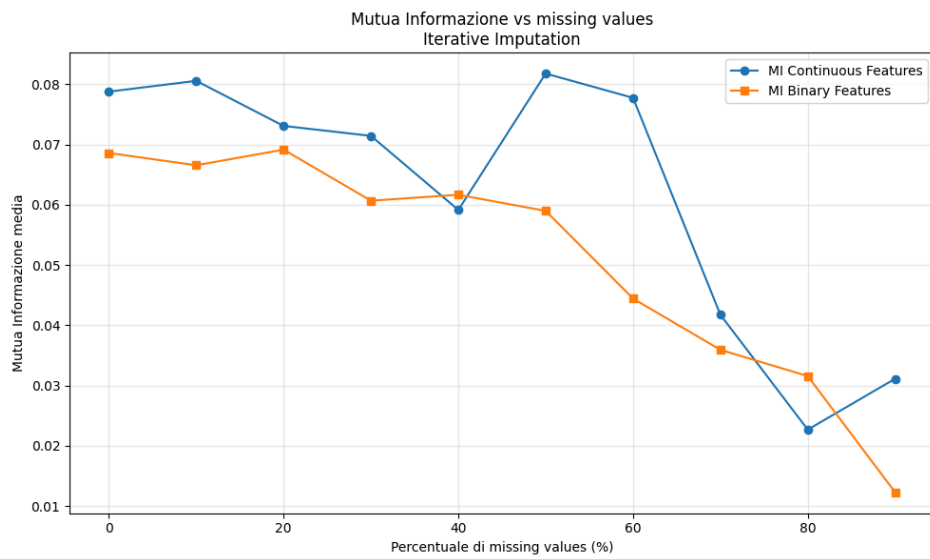
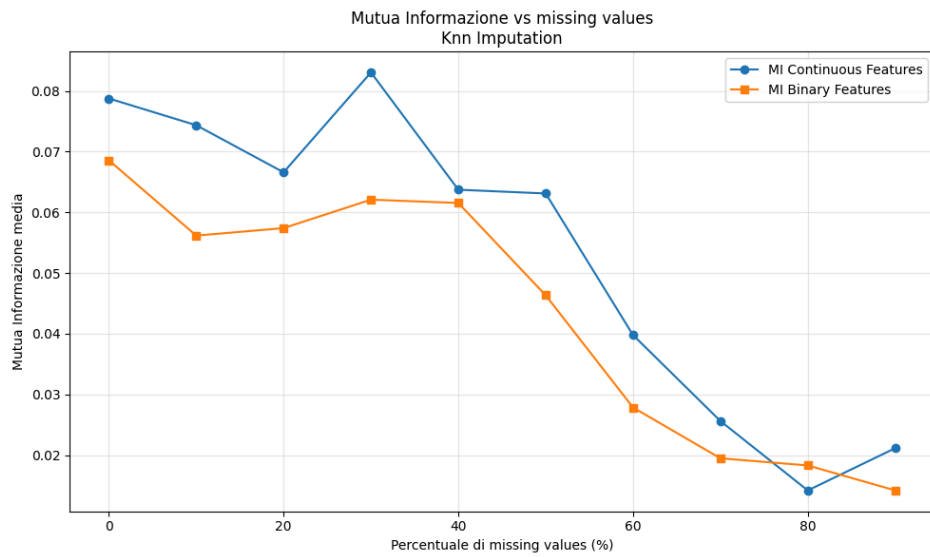
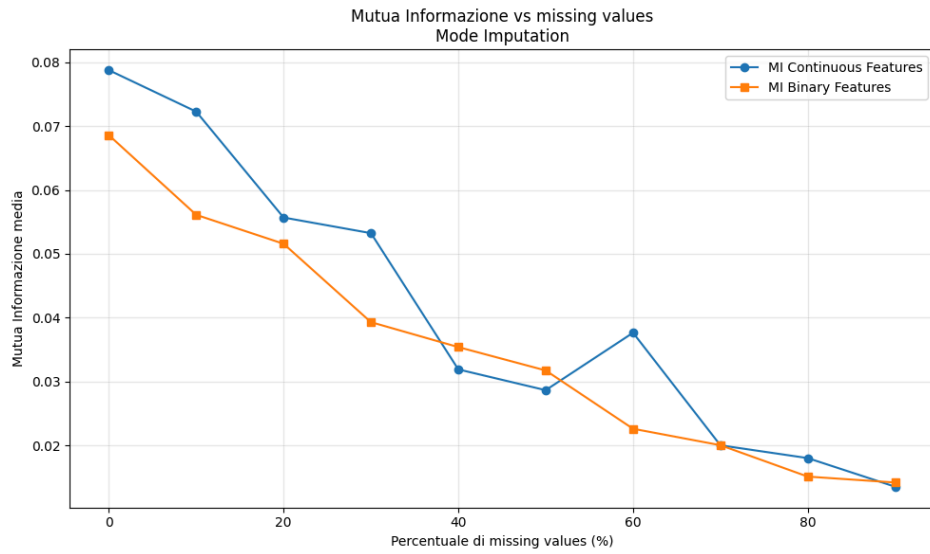




Anche per quanto riguarda gli alberi di decisione ottimizzati si ha che la feature più importante è ST_Slope_Up. Rispetto ai modelli naive si può osservare come ST_Slope_Up abbia un calo di rilevanza molto più contenuto rispetto ai modelli naive e questo potrebbe in parte spiegare le performance migliori ottenute dai modelli ottimizzati.

Inoltre per osservare come lo sporcamento del dataset vada ad impattare sulla qualità delle informazioni che riesce a fornire, ho creato dei grafici che mostrano l'andamento della mutua informazione al crescere della degradazione del dataset. Avendo creato un grafico singolo per metodo di imputazione riusciamo anche ad analizzare quale tra questi metodi permette di mantenere più informazione.





Dai grafici è possibile osservare come la mutua informazione cala al crescere del livello di degradazione del dataset, come prevedibile. Inoltre dai grafici si può osservare che i metodi di imputazione che mantengono più informazione sono knn e l'iterativo che pagano però la loro maggiore solidità con tempi di esecuzione molto più alti rispetto agli altri metodi. Quindi in conclusione la scelta del miglior metodo di imputazione dipende dalle proprie esigenze. Se si ha una grande percentuale di valori mancanti e si cerca le migliori performance possibili allora knn e il metodo iterativo risultano essere le scelte migliori, mentre se si hanno pochi valori nulli o si ricerca maggiore efficienza allora uno tra media, mediana e moda risulta essere il metodo di imputazione migliore.

valori duplicati

I valori duplicati sono un problema estremamente comune nei dataset del mondo reale. Le cause della loro presenza possono essere diverse: un semplice errore umano durante l'inserimento, un sistema che registra più volte la stessa misurazione o, ancora più spesso, un'integrazione imperfetta di dati provenienti da fonti diverse che non riconoscono le informazioni già presenti. Studiare il comportamento dei modelli di machine learning quando si introducono dati duplicati è cruciale per capirne la **robustezza**. I modelli non sono infallibili: un eccesso di righe identiche può ingannare l'algoritmo, che potrebbe erroneamente pensare che quelle informazioni siano più importanti di altre. Questo fenomeno può portare all'**overfitting**, dove il modello impara a memoria i dati ripetuti invece di catturare i pattern generali.

implementazione

Per implementare progressivamente i valori duplicati ho utilizzato la pipeline precedentemente descritta. La funzione che va a generare i duplicati è la seguente:

```
# Data Splitting and Augmentation
from sklearn.model_selection import train_test_split

# 1. DEFINE PROGRESSIVE DUPLICATE FUNCTION FIRST
def progressive_duplicate(df, steps=10, random_state=None):
    """
    Duplica progressivamente una percentuale crescente di righe del DataFrame.
    Restituisce una lista di DataFrame con duplicazioni crescenti.
    """
    np.random.seed(random_state)
    dfs = []
    dfs.append(df.copy()) # Step 0: Original Data

    n_rows = df.shape[0]
    for i in range(1, steps + 1):
        perc = i / steps
        n_dup = int(n_rows * perc)
```

```

        idx_to_duplicate = np.random.choice(df.index, n_dup, replace=False)
        df_dup = pd.concat([df, df.loc[idx_to_duplicate]], ignore_index=True)
        dfs.append(df_dup)
    return dfs

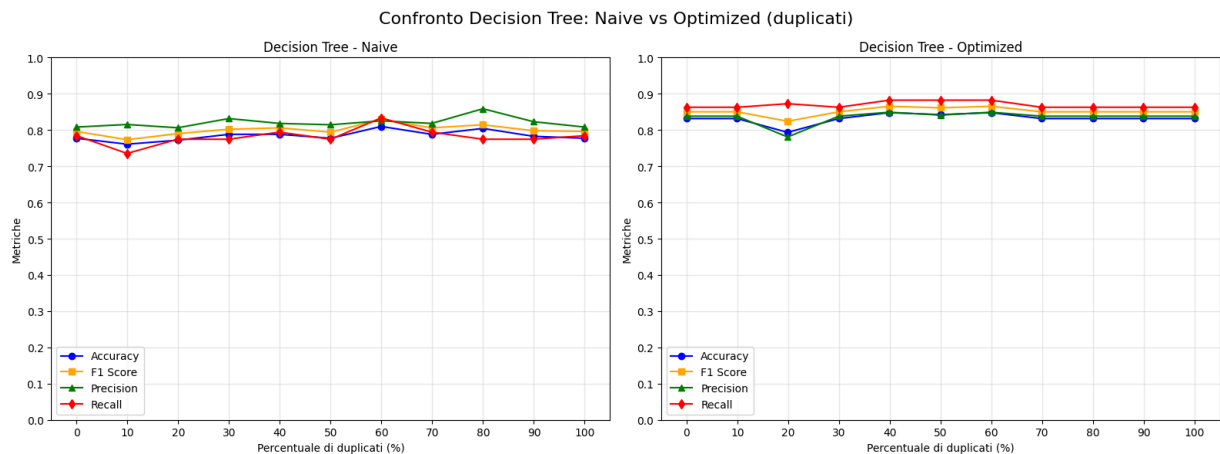
# 2. SPLIT FIRST (Crucial Step to prevent Leakage)
X = df.drop('HeartDisease', axis=1)
y = df['HeartDisease']
# Create the clean test set (20%) and the training set (80%)
X_train_orig, X_test_clean, y_train_orig, y_test_clean = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# Recombine X_train and y_train into a DataFrame
df_train = pd.concat([X_train_orig, y_train_orig], axis=1)
# 3. DUPLICATE ONLY THE TRAINING DATA
duplicated_dfs = progressive_duplicate(df_train, steps=10, random_state=42)

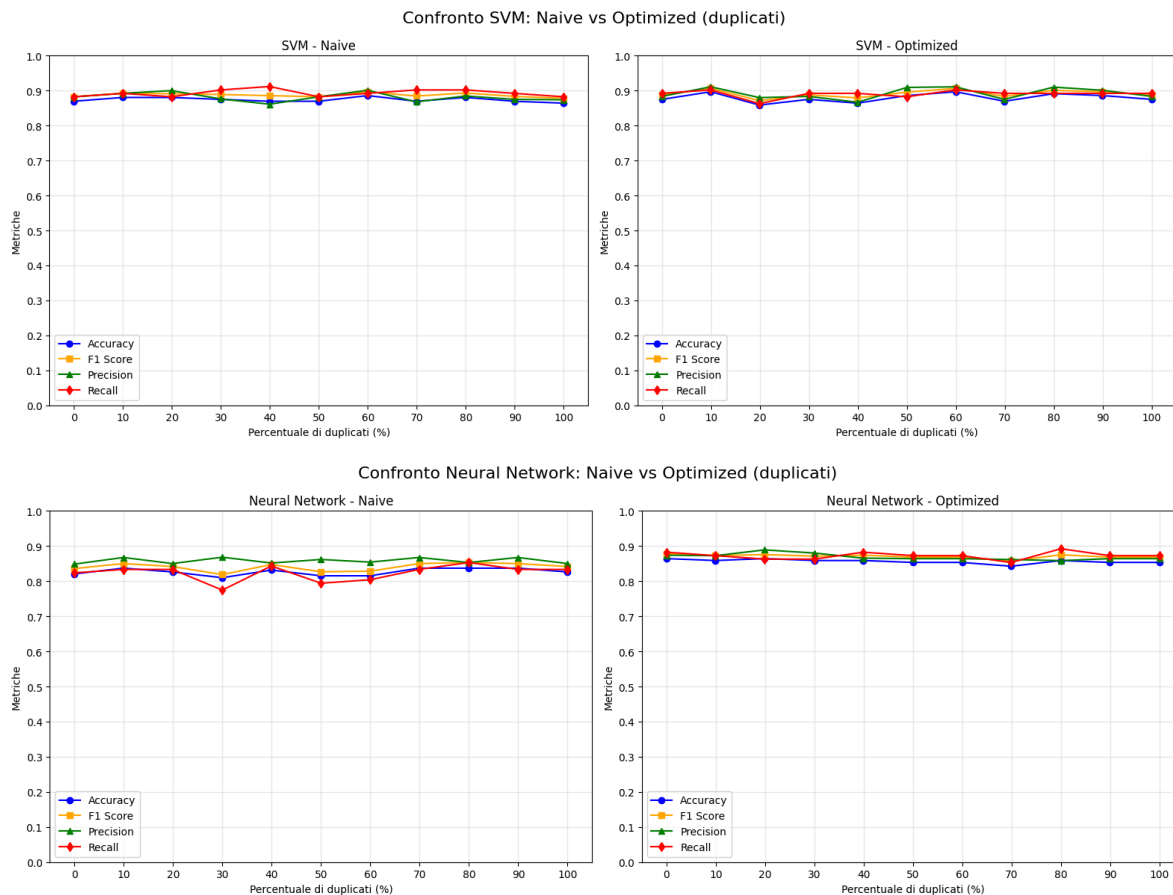
```

Questa funzione va a generare duplicati senza sovrascrivere i dati già presenti ma creandone di nuovi. In questo modo al termine del processo di degradazione otteniamo un dataset di dimensione doppia rispetto all'originale. Ho scelto questo approccio in quanto risulta essere quello più attinente alla realtà, infatti gli errori che portano a questa problematica vanno ad aggiungere nuove istanze al dataset senza sovrascrivere quelle già presenti (esempio: un'integrazione imperfetta di dati provenienti da fonti diverse che non riconoscono le informazioni già presenti).

Analisi risultati

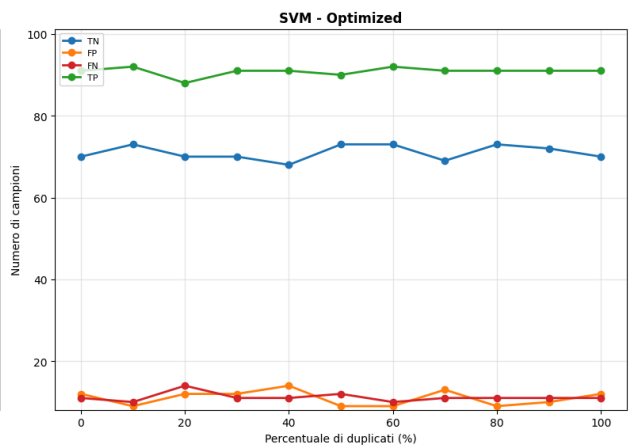
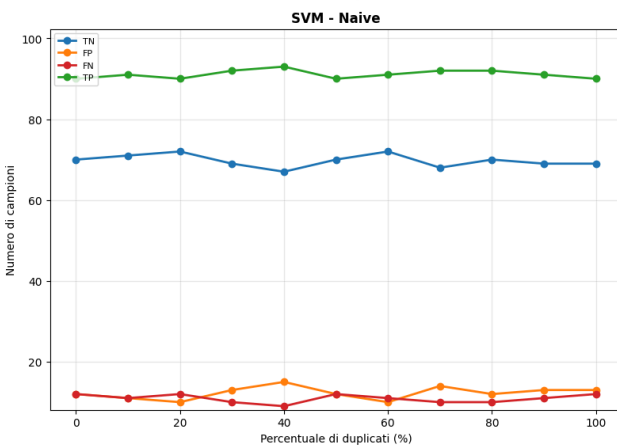
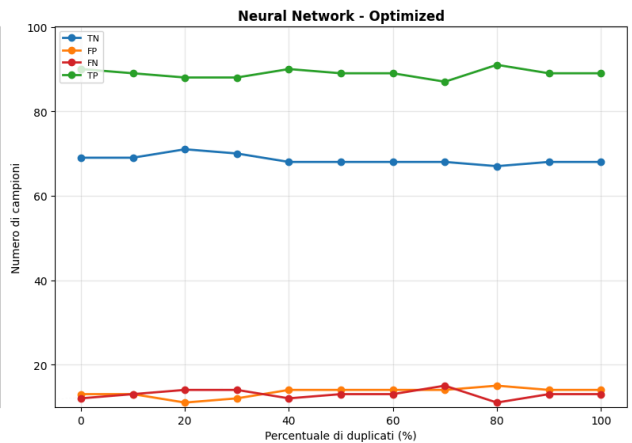
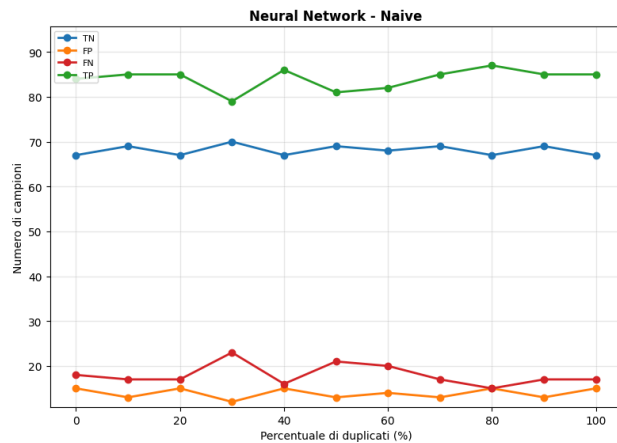
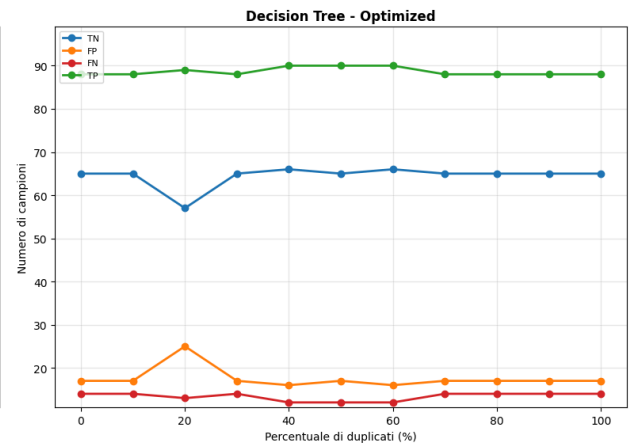
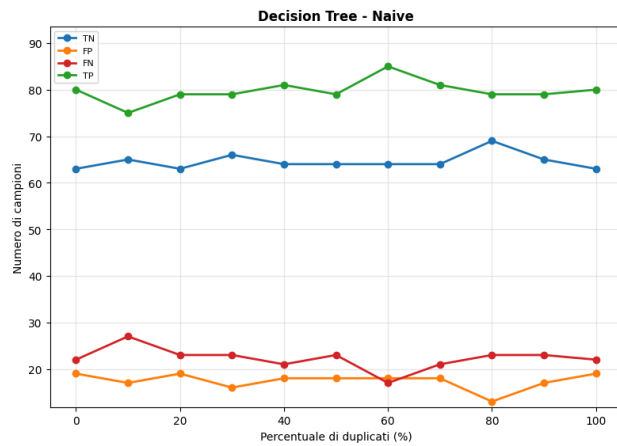
Per analizzare i risultati ho prodotto dei grafici che mostrano l'andamento delle metriche al crescere del livello di degradazione del dataset





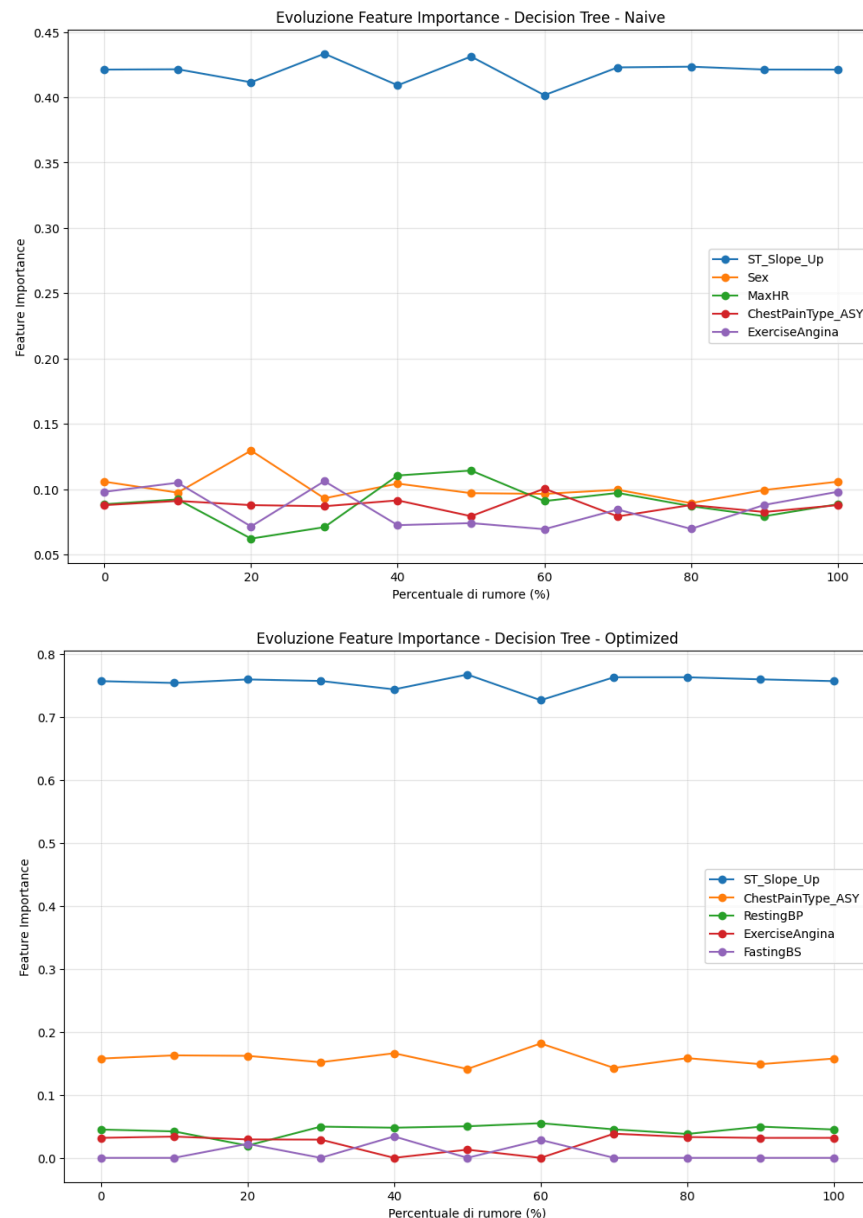
I risultati mostrano andamenti delle metriche pressoché costanti per tutti i modelli analizzati. Andando più nel dettaglio possiamo osservare come i modelli naive mostrino un andamento leggermente più irregolare delle metriche, mentre i modelli ottimizzati, in particolar modo SVM e le reti neurali, rimangono praticamente invariate per tutti gli step di degradazione del dataset. Questo può essere spiegato dall'utilizzo degli iperparametri che aiutano il modello a generalizzare meglio ed evitare di adattarsi troppo ai dati di training, particolarmente utile nel caso della presenza di duplicati.

Di seguito ho riportato i grafici che mostrano l'andamento delle matrici di confusione al crescere del numero di duplicati



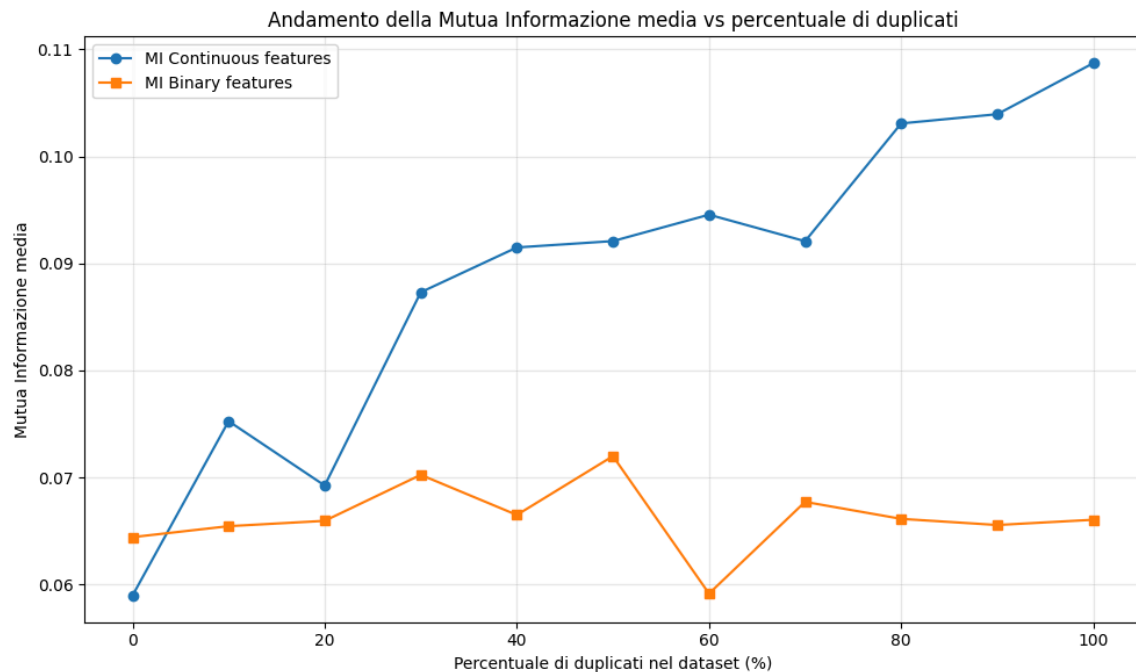
Da questi grafici si può osservare come i duplicati non abbiano causato grossi danni a livello di falsi positivi e falsi negativi, infatti questi seppur con andamenti leggermente irregolari, rimangono nel complesso piuttosto bassi.

Oltre ad analizzare le performance delle metriche sono andato a studiare anche l'evoluzione dell'importanza delle feature negli alberi di decisione, per comprendere meglio le cause dell'andamento delle metriche



Dai grafici risulta ancora che la feature più importante negli alberi di decisione è la ST_Slope_Up. Il degradamento del dataset con i duplicati va ad intaccare la scelta delle feature in maniera significativa solo negli alberi di decisione naive in cui si può osservare un andamento leggermente irregolare dell'importanza delle feature. Per quanto riguarda invece l'albero di decisione ottimizzato l'importanza delle feature non sembra essere intaccata dal numero di duplicati presenti nel dataset.

Inoltre per osservare come lo sporcamento del dataset vada ad impattare sulla sua qualità nel dare informazioni significative ho creato dei grafici che mostrano l'andamento della mutua informazione al crescere della degradazione del dataset.



La mutua informazione delle feature continue cresce in maniera direttamente proporzionale al numero di duplicati aggiunti, seppur con un andamento irregolare. Invece la mutua informazione delle feature binarie ha un andamento irregolare in cui però non si registrano particolare miglioramenti per quanto riguarda la mutua informazione. La differenza osservata può essere spiegata considerando la natura delle due tipologie di variabili. Le **feature continue**, avendo un range di valori molto ampio, quando vengono duplicate rafforzano le relazioni già presenti con la variabile target: la ripetizione di determinati valori numerici aumenta la probabilità congiunta di specifiche coppie (x,y), rendendo più evidente la dipendenza statistica. Questo si traduce in un incremento progressivo e regolare della mutua informazione al crescere dei duplicati. Le **feature binarie**, invece, possono assumere solo due stati (0 o 1). La duplicazione di queste osservazioni non modifica le frequenze relative delle due categorie e, di conseguenza, non aggiunge nuova informazione rispetto al target. Per questo motivo la mutua informazione rimane sostanzialmente stabile, con eventuali piccole oscillazioni dovute a effetti di stima o di campionamento, ma senza veri miglioramenti in termini di capacità predittiva.

outliers

Gli outlier sono valori che si discostano notevolmente dagli altri dati, e la loro presenza può influenzare negativamente l'analisi e la modellazione. Le ragioni della loro comparsa sono molteplici:

- **Errori di misurazione:** Un sensore malfunzionante o un errore di immissione dati può registrare un valore completamente fuori scala.
- **Variazioni naturali ma rare:** Alcuni outlier possono essere dati legittimi ma estremamente rari, come una transazione finanziaria insolitamente grande o un evento meteorologico estremo.

Analizzare il comportamento di un modello all'aumentare del numero di outliers fornisce una comprensione profonda della sua robustezza e del suo comportamento in condizioni reali.

Implementazione

Per implementare progressivamente gli outliers ho utilizzato la pipeline precedentemente descritta. La funzione che va a generare gli outliers è la seguente:

```
def add_continuous_outliers(self, contamination_rate=0.02, methods=['extreme'],
seed=None):
    """Contamina singoli valori numerici"""
    rng = np.random.default_rng(seed)
    if not self.continuous_features:
        return {}

    univariate_methods = [m for m in methods if m != 'multivariate']
    if not univariate_methods:
        univariate_methods = ['extreme']

    # Calcola quanti nuovi valori contaminare
    n_new_outliers = int(round(self.total_numeric_values *
contamination_rate))

    if n_new_outliers == 0 and contamination_rate > 0:
        n_new_outliers = 1

    # Genera tutte le possibili combinazioni (row_label, feature)
    all_value_positions = []
    for row_label in self.current_df.index:
        for feature in self.continuous_features:
            all_value_positions.append((row_label, feature))

    # Filtra quelle già contaminate
    available_positions = [pos for pos in all_value_positions
                           if pos not in self.contaminated_values]

    if len(available_positions) == 0:
        print("No more clean values available for contamination.")
        return {}

    if len(available_positions) < n_new_outliers:
        n_new_outliers = len(available_positions)
        print(f"Warning: Only {len(available_positions)} clean values
remaining. Using all of them.")

    # Scegli casualmente le posizioni da contaminare
    chosen_positions = rng.choice(len(available_positions),
size=n_new_outliers, replace=False)
    chosen_positions = [available_positions[i] for i in chosen_positions]

    contamination_info = {}

    # Assicurati di poter ripristinare i valori originali
    if not hasattr(self, 'original_values'):
```

```

self.original_values = {}

for row_label, feature in chosen_positions:
    method = str(rng.choice(univariate_methods))

    # Salva il valore originale (prima volta)
    if row_label not in self.original_values:
        self.original_values[row_label] = {}
    if feature not in self.original_values[row_label]:
        self.original_values[row_label][feature] =
self.current_df.at[row_label, feature]

    # Dati puliti: escludi valori già contaminati per questa feature
    clean_positions = [(rl, f) for rl, f in all_value_positions
                        if f == feature and (rl, f) not in
self.contaminated_values]
    clean_data = pd.Series([self.current_df.at[rl, f] for rl, f in
clean_positions]).dropna()

    if clean_data.empty:
        continue

    q1, q3 = clean_data.quantile(0.25), clean_data.quantile(0.75)
    iqr = q3 - q1
    std_val = float(clean_data.std()) if clean_data.std() != 0 else 1.0
    min_val, max_val = clean_data.min(), clean_data.max()

    orig_val = float(self.current_df.at[row_label, feature])

    # Genera l'outlier
    if method == 'extreme':
        if rng.random() < 0.5:
            outlier_val = q1 - rng.uniform(3, 6) * iqr
            outlier_val = max(outlier_val, float(min_val) - 3 * std_val)
        else:
            outlier_val = q3 + rng.uniform(3, 6) * iqr
            outlier_val = min(outlier_val, float(max_val) + 3 * std_val)
    elif method == 'shift':
        shift_factor = rng.choice([-1, 1]) * rng.uniform(2, 4)
        outlier_val = orig_val + shift_factor * std_val
    elif method == 'noise':
        noise_factor = rng.choice([-1, 1]) * rng.uniform(3, 5)
        outlier_val = orig_val + noise_factor * std_val
    else:
        # fallback
        outlier_val = q3 + rng.uniform(3, 6) * iqr

    # Domain-aware clipping
    if feature in ['Age', 'RestingBP', 'Cholesterol', 'MaxHR']:
        lower_bound = 0
        outlier_val = max(outlier_val, lower_bound)

    # Preserva il tipo intero se necessario
    if pd.api.types.is_integer_dtype(self.current_df[feature].dtype):
        outlier_val = int(round(outlier_val))

```

```

# Applica la modifica
self.current_df.at[row_label, feature] = outlier_val

# Registra l'informazione
contamination_info.setdefault(feature, []).append({
    'index': row_label,
    'original_value': self.original_values[row_label][feature],
    'outlier_value': outlier_val,
    'method': method
})

# Aggiorna il set dei valori contaminati
self.contaminated_values.update(set(chosen_positions))

return contamination_info

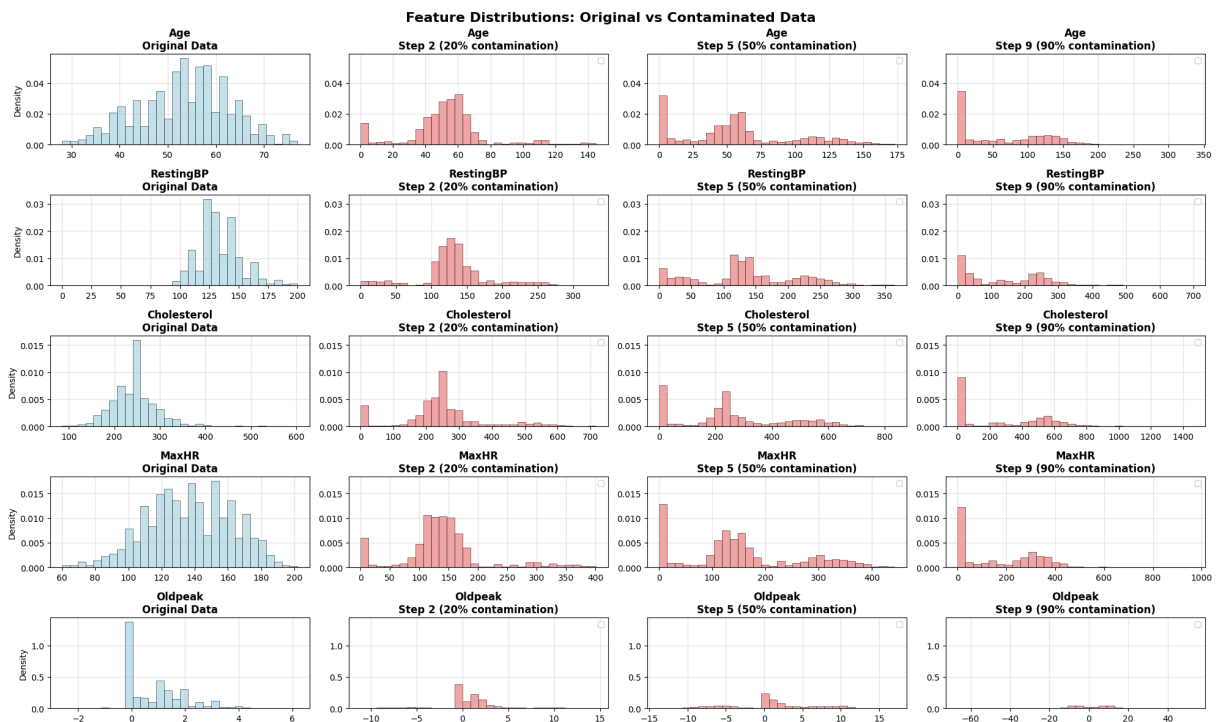
```

Questa funzione genera outliers per le feature numeriche presenti nel dataset. Vengono generati 3 diversi tipi di outliers:

- **extreme:** produce valori che si discostano molto dalla distribuzione iniziale e sono utili per simulare gli errori più evidenti come malfunzionamenti di sensori o errore di inserimento dati. Questi outliers vengono generati utilizzando il metodo del range interquartile (IQR). Questa tecnica statistica per identificare valori anomali si sviluppa nel seguente modo:
 - a. Si calcolano i quartili
 - b. Si calcola l'interquartile (IQR) ovvero la differenza tra il terzo e il primo quartile. Questo valore rappresenta la “diffusione” dei dati centrali
 - c. Si calcolano i limiti oltre i quali i valori vengono considerati outliers. Limite inferiore: $Q1 - 1.5 \cdot IQR$. Limite superiore: $Q3 + 1.5 \cdot IQR$
 - d. Vengono generati valori casuali che si trovano al di fuori di questi confini
- **shift:** questo metodo sposta il valore originale di un multiplo della deviazione standard. Risulta utile in quanto simula scenari come l'errore di conversione dell'unità di misura
- **noise:** Questo metodo aggiunge un rumore casuale ma significativo al valore originale. L'entità del rumore è sempre un multiplo della deviazione standard. Questo metodo è particolarmente utile per simulare anomalie che si verificano naturalmente, dove i dati non sono intrinsecamente "sbagliati", ma sono influenzati da una forte variabilità o da misurazioni imprecise. Questo è comune in dati finanziari o medici. Studiare questo tipo di outlier aiuta a testare la resilienza dei modelli a un rumore di fondo più realistico.

Analisi risultati

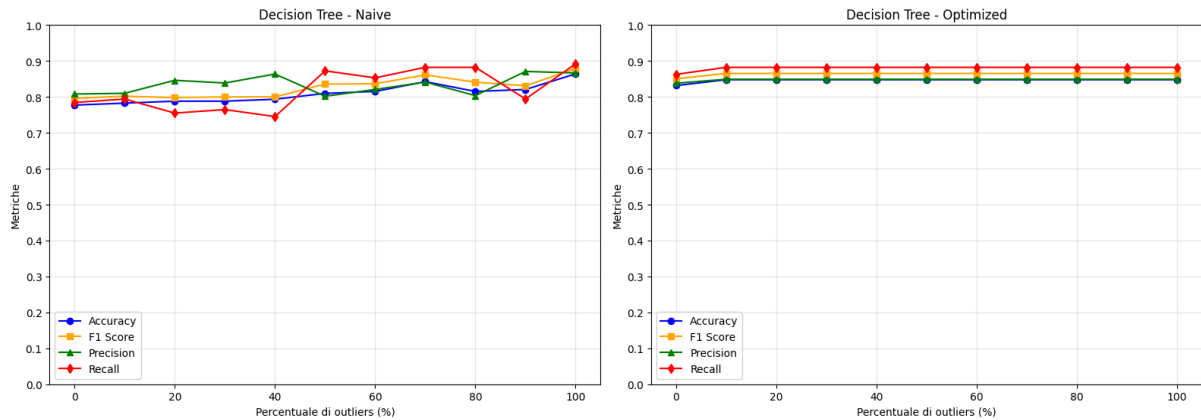
Per prima cosa ho creato dei grafici per visualizzare gli outliers inseriti e come andassero ad impattare sulla distribuzione dei dati iniziale



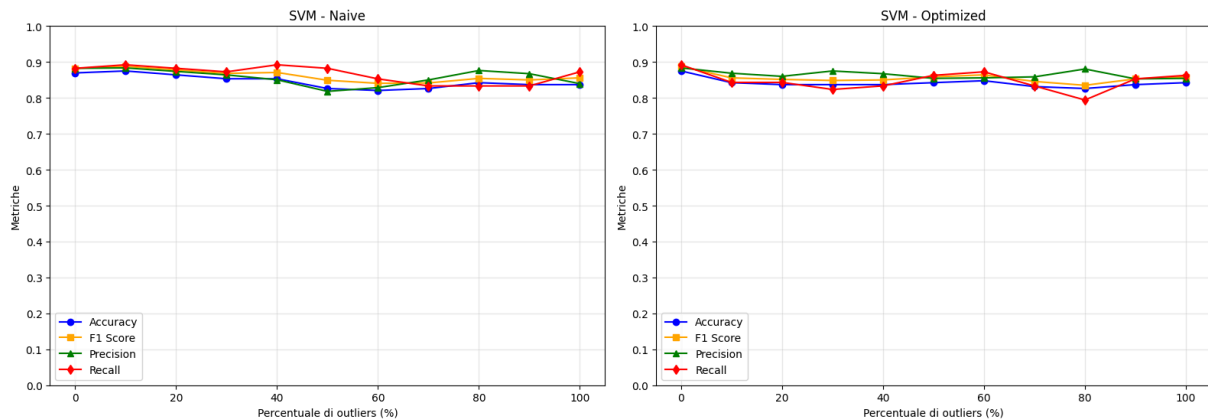
Dai grafici risulta che l'inserimento di outliers, come prevedibile, ha portato ad una forte modifica della distribuzione iniziale passando da distribuzioni normali a distribuzioni molto più appiattite e disperse. Osservando l'evoluzione delle distribuzioni delle feature numeriche ai diversi step di contaminazione emerge che in molte variabili gli outlier tendono a concentrarsi verso i valori inferiori, spesso in prossimità dello zero. Questo fenomeno può essere spiegato considerando sia la natura del dataset originale, sia dai vincoli applicati durante la generazione degli outlier. Le variabili analizzate, come ad esempio Age, RestingBP, Cholesterol e MaxHR, presentano valori strettamente positivi e distribuiti in intervalli relativamente ridotti. Quando il metodo di contaminazione produce valori estremi sottraendo multipli dell'intervallo interquartile dal primo quartile, il risultato porta naturalmente alla generazione di valori inferiori a zero. Tuttavia, il codice di contaminazione impone un vincolo che impedisce a queste feature di assumere valori negativi, bloccandole al valore minimo consentito pari a zero. La combinazione di queste due condizioni, ossia la tendenza della procedura a spingere alcuni valori verso il basso e la correzione automatica che impedisce di superare il limite inferiore, determina un accumulo artificiale di outlier intorno a zero. In questo modo si spiegano i picchi evidenziati nei grafici in corrispondenza dei valori più bassi delle distribuzioni contaminate.

Successivamente sono andato a creare i grafici per mostrare l'andamento delle metriche al crescere del numero di outliers

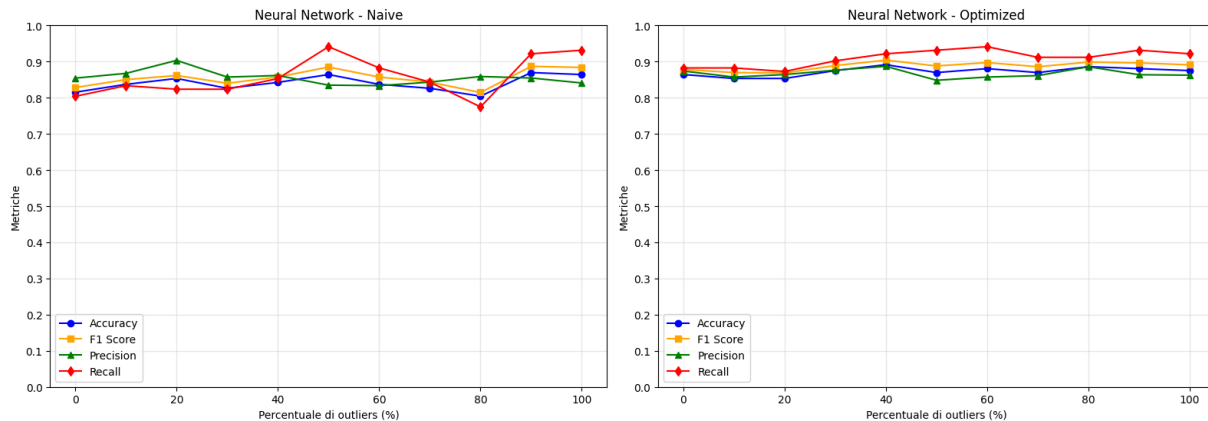
Confronto Decision Tree: Naive vs Optimized



Confronto SVM: Naive vs Optimized



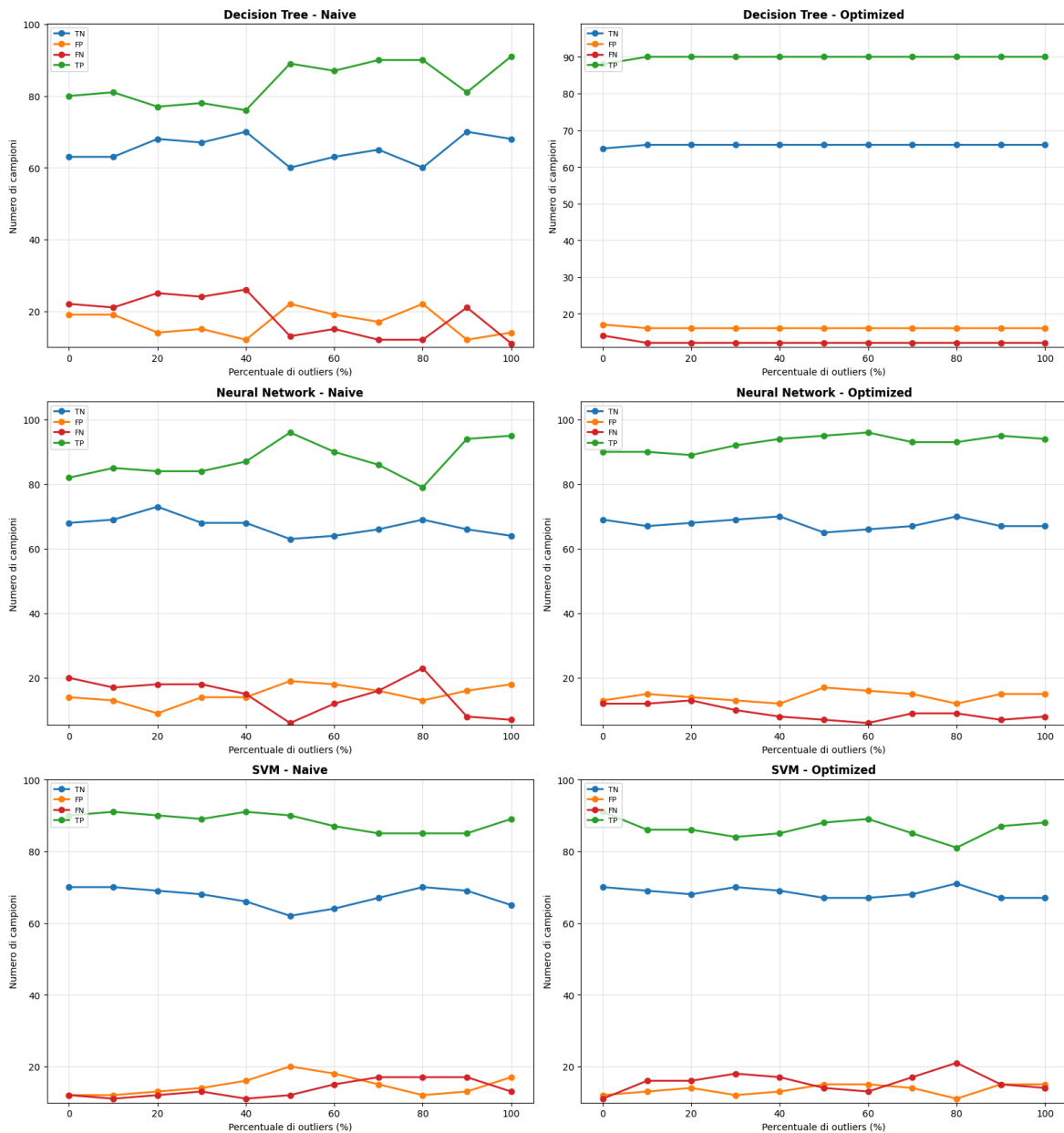
Confronto Neural Network: Naive vs Optimized



Dai grafici si osserva come l'inserimento progressivo di outliers non causi un crollo netto delle performance, ma solo un po' di instabilità delle metriche, in particolare nei modelli naive. Invece i modelli ottimizzati, ancora una volta, mostrano performance molto solide delle metriche, senza particolare cali di performance. In particolare si osserva come l'albero di decisione ottimizzato ha registrato performance praticamente identiche al dataset pulito. Questo comportamento può essere spiegato dal fatto che l'albero ottimizzato utilizzi quasi unicamente feature binarie per fare previsione e non usi feature continue in maniera significativa. Essendo che le uniche feature degradate con gli outliers sono state quelle continue questo risultato appare completamente giustificato.

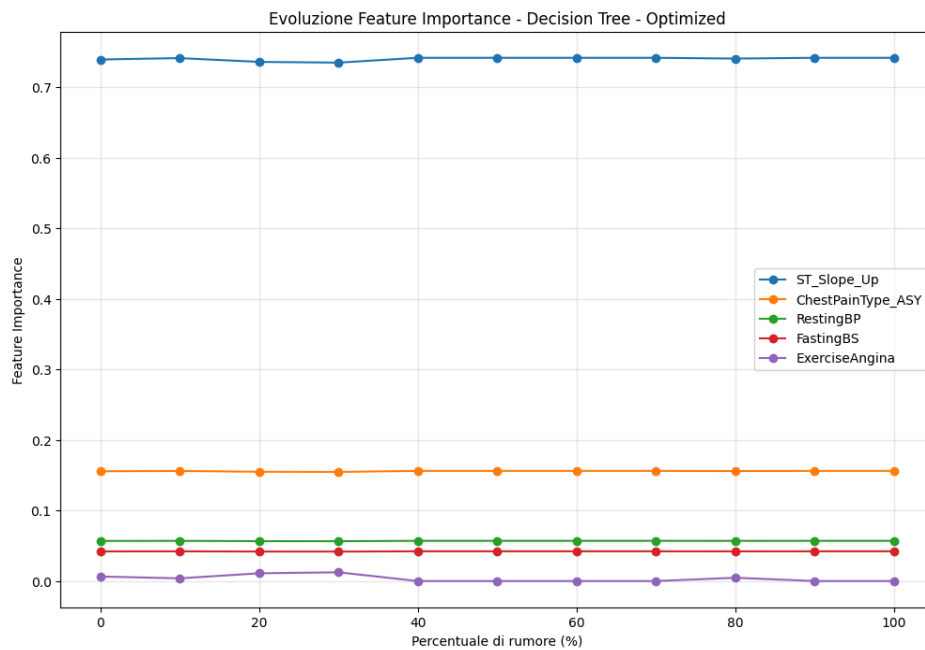
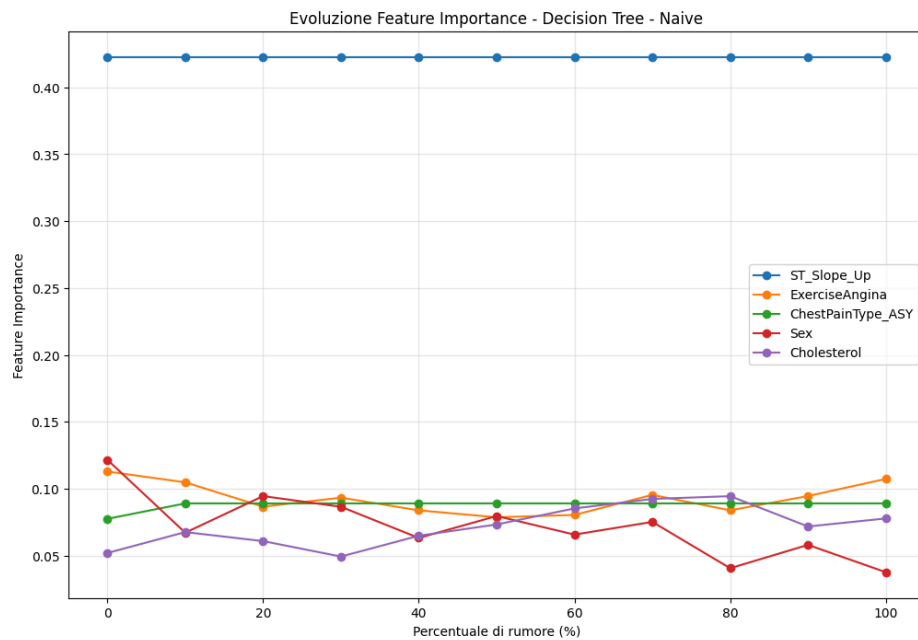
In generale, questo tipo di rumore ha un impatto limitato sulle performance complessive dei modelli, soprattutto perché il dataset contiene prevalentemente feature binarie e una quota minore di feature continue. Questo squilibrio nella tipologia di dati può giustificare il comportamento osservato nei grafici.

Di seguito sono presenti i grafici contenenti l'evoluzione delle matrici di confusione al crescere del numero di outliers



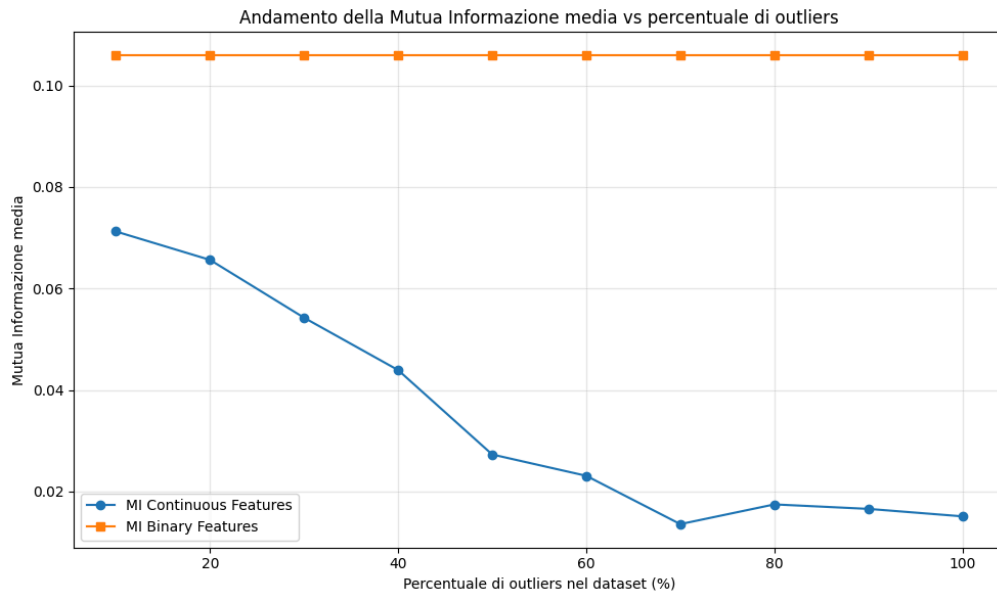
Da questi grafici si può osservare come gli outliers non abbiano causato grossi danni a livello di falsi positivi e falsi negativi, infatti questi seppur con andamenti irregolari, rimangono nel complesso piuttosto bassi.

I seguenti grafici riportano l'evoluzione dell'importanza delle feature al crescere del numero di outliers inseriti nel dataset



Questi grafici confermano quanto detto in precedenza, infatti si può osservare come l'importanza delle feature nell'albero ottimizzato rimangano praticamente sempre costanti. Questo è dovuto al fatto che utilizzi quasi unicamente feature binarie che non sono intaccate dagli outliers. Invece nell'albero naive si possono osservare variazioni nell'importanza delle feature a causa della presenza di feature numeriche come Cholesterol.

Infine riporto anche i grafici sulla variazione della mutua informazione al crescere del numero di outliers



Da questo grafico si osserva come la qualità delle feature numeriche vada a calare al crescere del numero di outliers. Invece la mutua informazione delle feature binarie rimane costante in quanto non vengono modificate dal processo di inserimento di outliers.

rumore su feature binarie

Il rumore sulle feature binarie, anche detto flip noisy, è un problema comune all'interno dei dataset. Può essere causato da errori umani nel data entry, etichettature imprecise da parte di sistemi automatici o annotazioni scorrette in grandi dataset. Studiare l'impatto di questo rumore è utile per cercare di capire quali tipi di modelli sono più solidi rispetto a questo tipo di problematica.

implementazione

Per implementare progressivamente il flip noisy sulle feature binarie ho utilizzato la pipeline precedentemente descritta. La funzione che va a generare il rumore è la seguente:

```
def add_progressive_noise_to_binary_features(df: pd.DataFrame, binary_features: list,
target_column: str = None) -> dict:
    # Filtra le colonne binarie escludendo la colonna target se specificata
    columns_to_noise = [col for col in binary_features if col != target_column] if target_column
    else binary_features

    if len(columns_to_noise) == 0:
        raise ValueError("Nessuna colonna binaria da modificare (tutte escluse o lista vuota).")

    print(f"Colonne binarie da modificare: {columns_to_noise}")

    noisy_datasets = {}

    # Calcola il numero totale di elementi binari nel dataset
    total_binary_elements = len(df) * len(columns_to_noise)
    print(f"Totale elementi binari da considerare: {total_binary_elements}")

    # Itera attraverso le percentuali di rumore da 0 a 100 con step di 10
```

```

for percentage in range(0, 101, 10):
    # Lavoriamo sempre su una copia pulita del df originale
    df_noisy = df.copy()

    # Se la percentuale è 0, non facciamo nulla
    if percentage == 0:
        noisy_datasets[f'0%_noise'] = df_noisy
        print(f"Generato dataset con 0% di rumore (nessuna modifica).")
        continue

    # Calcola il numero totale di elementi da invertire
    n_to_flip = int(np.floor(total_binary_elements * (percentage / 100)))

    # Crea una lista di tutte le posizioni possibili (riga, colonna)
    all_positions = [(row_idx, col) for row_idx in df.index for col in columns_to_noise]

    # Seleziona casualmente n_to_flip posizioni da invertire
    # Usiamo random_state per riproducibilità
    np.random.seed(42)
    positions_to_flip = np.random.choice(len(all_positions), size=n_to_flip, replace=False)

    # Inverti i valori nelle posizioni selezionate
    for pos_idx in positions_to_flip:
        row_idx, col = all_positions[pos_idx]
        df_noisy.loc[row_idx, col] = 1 - df_noisy.loc[row_idx, col]

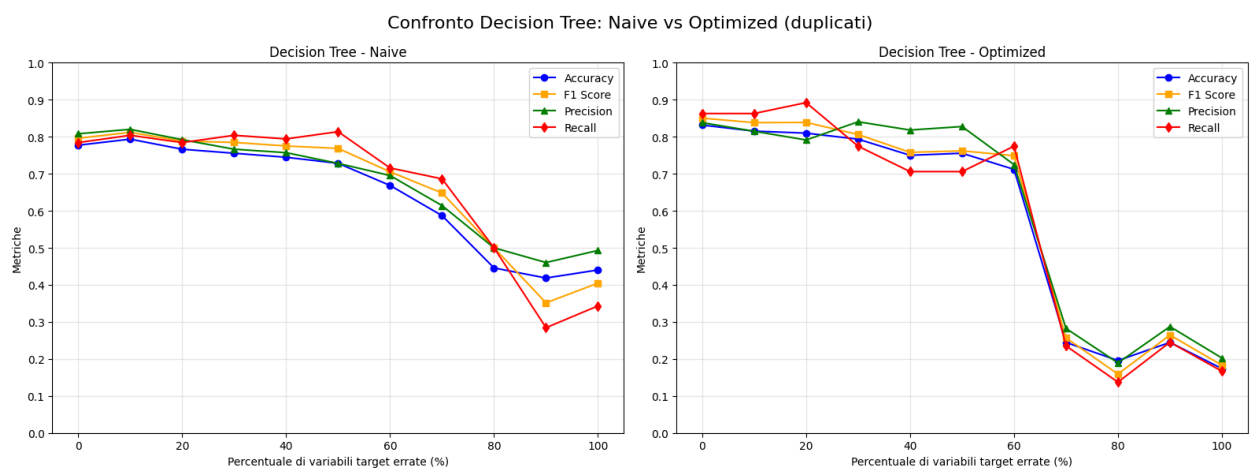
    # Salva il DataFrame rumoroso nel dizionario
    key_name = f'{percentage}%_noise'
    noisy_datasets[key_name] = df_noisy
    print(f"Generato dataset con {percentage}% di rumore ({n_to_flip} elementi binari invertiti).")

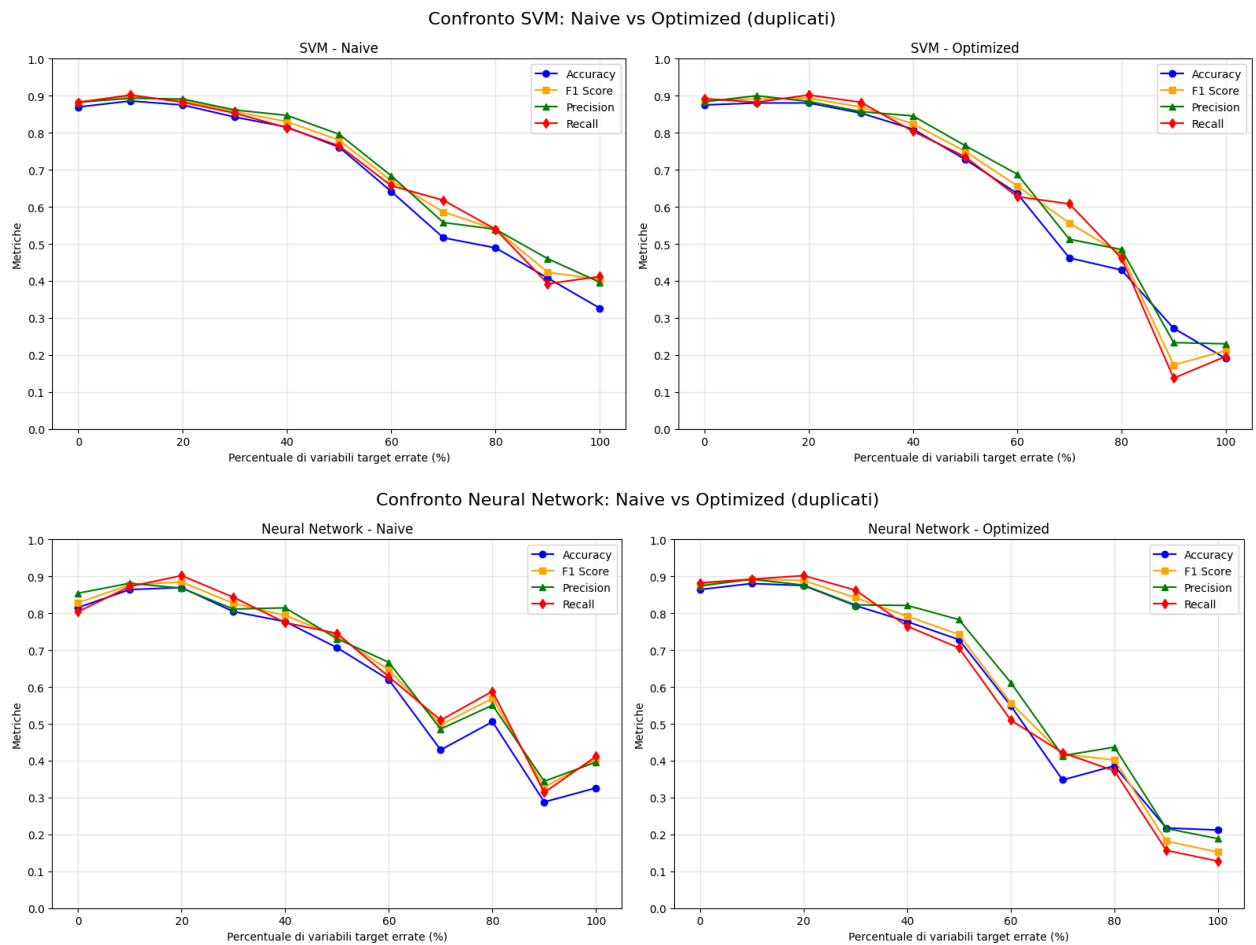
return noisy_datasets

```

Analisi risultati

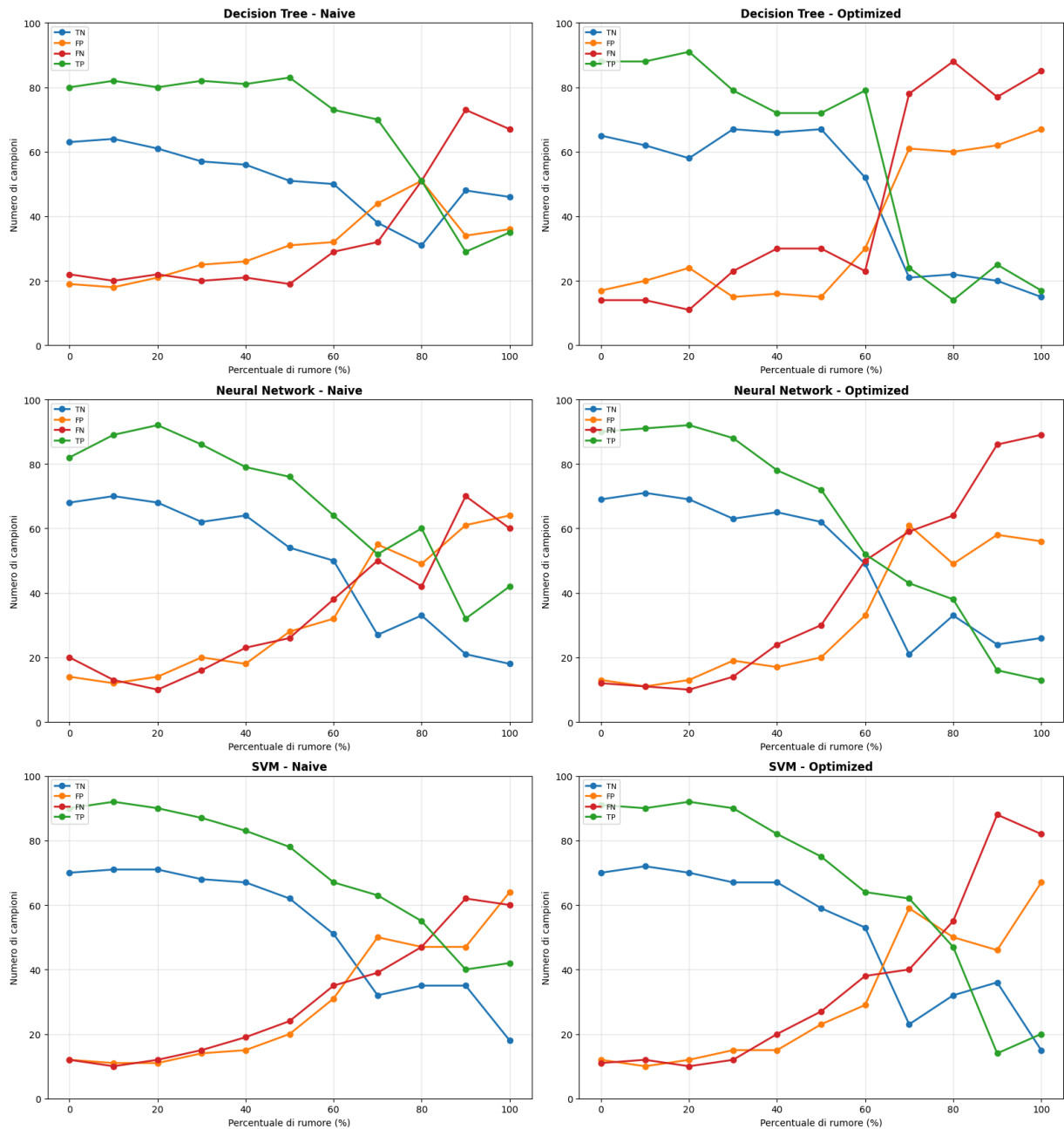
Come prima cosa sono andato a visualizzare l'andamento delle metriche al crescere della percentuale di rumore aggiunta alla feature binarie:





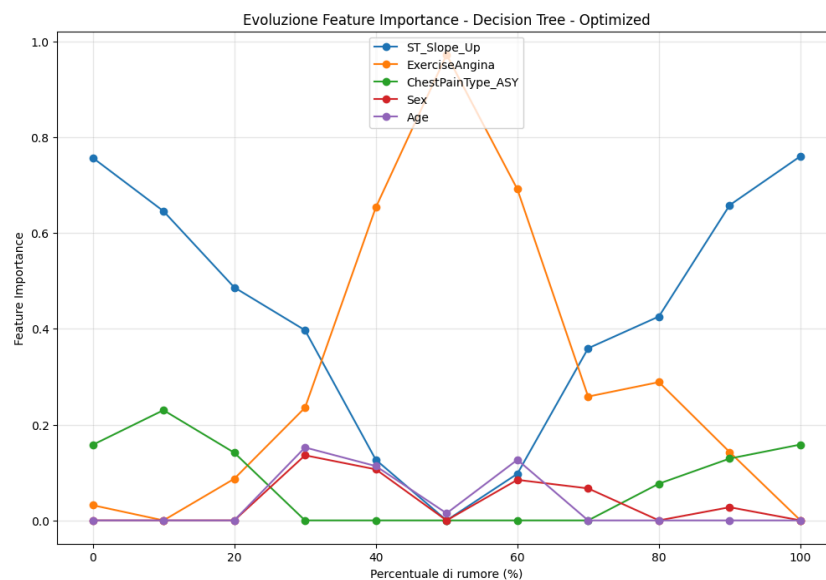
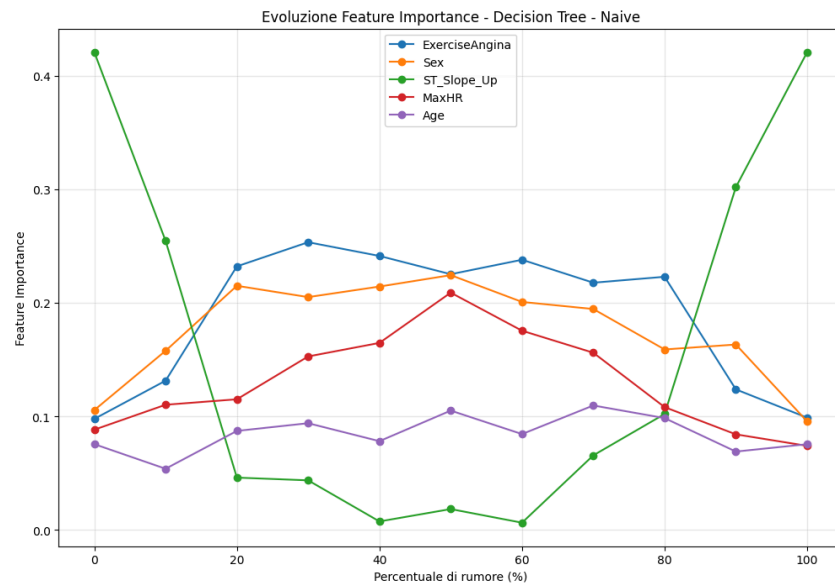
Dai grafici risulta come questa tipologia di rumore impatti in maniera significativa le performance di tutti i modelli analizzati, anche nelle loro versioni ottimizzate. Più in dettaglio abbiamo che i modelli registrano performance piuttosto solide con percentuali di rumore inferiori al 30%, dopo di che si ha una discesa più o meno rapida in tutti i modelli, con il picco di peggioramento osservabile oltre il 60% di rumore. SVM mantiene curve più regolari e resistenti fino a circa il 50–60% di rumore in entrambe le versioni dopo di che si registra un'accelerazione del peggioramento delle performance. Un comportamento praticamente sovrapponibile è stato registrato dalle reti neurali. L'albero di decisione invece, presenta nella versione naive un peggioramento graduale delle performance, mentre nella versione ottimizzata si registra un crollo netto delle performance tra il 60-70%.

Di seguito invece riporto l'andamento delle matrici di confusione al crescere del livello di degradazione delle feature binarie.



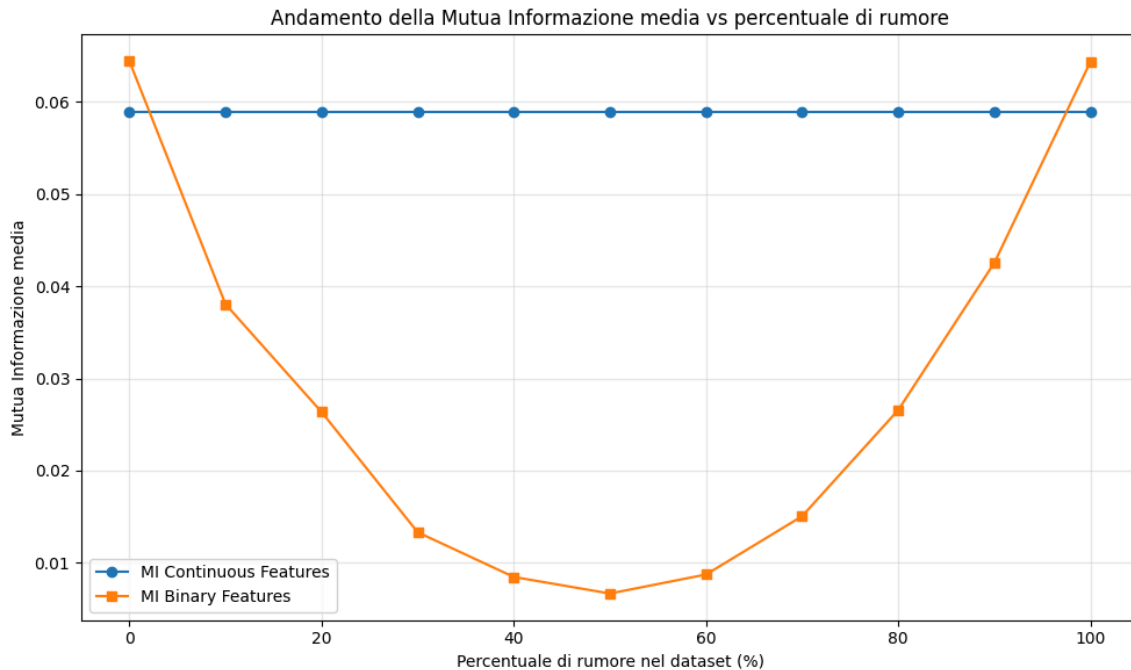
Dai grafici si osserva come i modelli più robusti sono SVM e le reti neurali in cui si registrano performance paragonabili, con valori accettabili di FN e FP sotto alla soglia del 60% di rumore.

Di seguito ho invece riportato l'evoluzione dell'importanza delle feature negli alberi di decisione



Nell'albero di decisione ottimizzato ST_Slope_Up domina a rumore basso e torna rilevante anche oltre 80–100% di rumore, segno che resta la variabile più stabile e informativa quando il modello trova split affidabili; nel mezzo (circa 40–60%) emerge ExerciseAngina con un picco molto marcato, indicando che, quando il rumore erode la discriminatività di ST_Slope_Up, l'albero sposta gli split su un feature ancora utile. Nell'albero di decisione naive invece l'importanza delle feature risulta molto più distribuita con ST_Slope_Up che cala bruscamente già dai primi step, per poi ricrescere in maniera improvvisa con percentuali di rumore elevate (80-100%)

Infine riporto anche i grafici sulla mutua informazione registrata al crescere del livello di degradazione delle feature binarie.



Dal grafico si osserva correttamente come la mutua informazione delle feature continue non venga intaccata da questa tipologia di rumore. Invece per quanto riguarda la mutua informazione delle feature binarie si osserva un curva a U. Questo è esattamente ciò che ci si aspetta con flip noise: più ci si avvicina al 50% di flip, più la relazione con il target diventa casuale e la mutua informazione tende a zero; superato quel punto, proseguendo ad invertire ci si allo stato originale fino ad arrivare al 100% in cui si hanno le feature binarie perfettamente invertite, ripristinando il valore originale della mutua informazione

rumore su feature target

Il rumore della variabile target è un problema comune. Può essere causato da errori umani nel data entry, diagnosi mediche sbagliate, etichettature imprecise da parte di sistemi automatici o annotazioni scorrette in grandi dataset. Studiare l'impatto di questo rumore è utile per cercare di capire quali tipi di modelli sono più solidi rispetto a questo tipo di problematica.

implementazione

Per implementare progressivamente il rumore sulla variabile target ho utilizzato la pipeline precedentemente descritta. La funzione che va a generare il rumore è la seguente

```
def add_progressive_noise(df: pd.DataFrame, target_column: str) -> dict:
    # Controlla che la colonna target sia binaria (contenente solo 0 e 1)
    if not all(df[target_column].isin([0, 1])):
        raise ValueError(f"La colonna '{target_column}' non è binaria (deve contenere solo 0 e 1).")

    noisy_datasets = {}
    n_samples = len(df)

    print(f"DataFrame originale con {n_samples} righe.")
```

```

# Itera attraverso le percentuali di rumore da 0 a 100 con step di 10
for percentage in range(0, 101, 10):
    # Lavoriamo sempre su una copia pulita del df originale per evitare
    # rumore cumulativo
    df_noisy = df.copy()

    # Se la percentuale è 0, non facciamo nulla e salviamo la copia
    # originale
    if percentage == 0:
        noisy_datasets[f'0%_noise'] = df_noisy
        continue

    # Calcola il numero di etichette da "flippare" (invertire)
    n_to_flip = int(np.floor(n_samples * (percentage / 100)))

    # Seleziona casualmente gli indici delle righe da modificare
    # Usiamo random_state per la riproducibilità degli esperimenti
    indices_to_flip = df.sample(n=n_to_flip, random_state=42).index

    # Inverti i valori nella colonna target per gli indici selezionati
    df_noisy.loc[indices_to_flip, target_column] =
~df_noisy.loc[indices_to_flip, target_column]

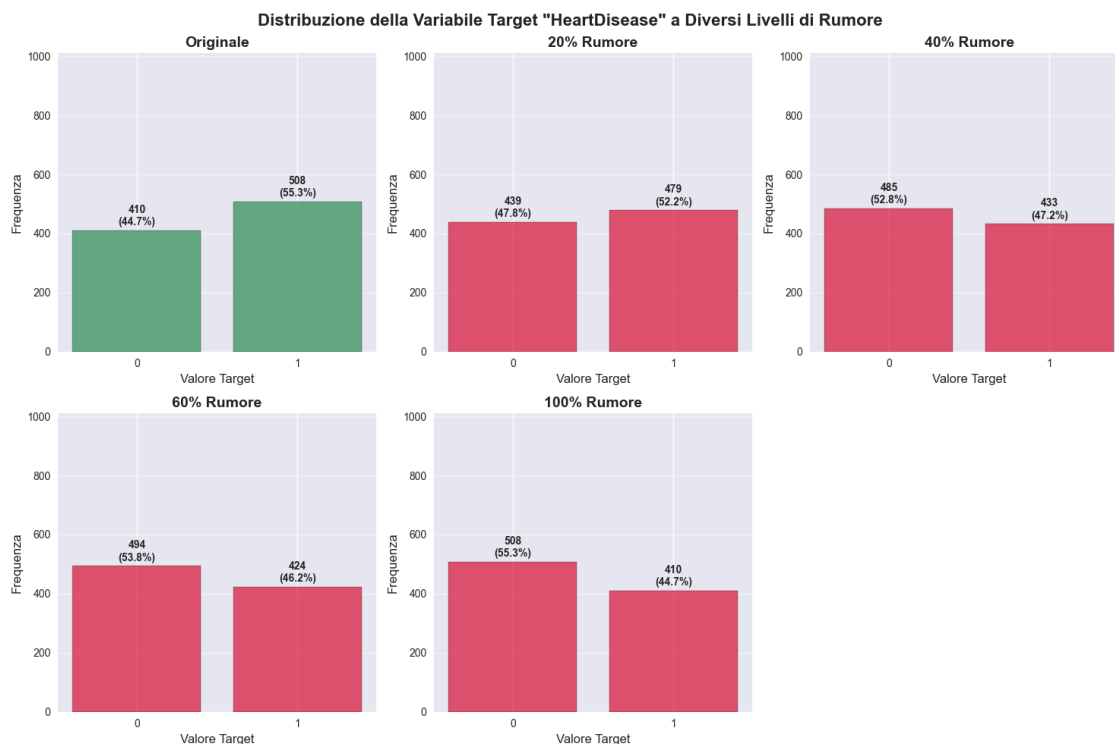
    # Salva il DataFrame rumoroso nel dizionario
    key_name = f'{percentage}%_noise'
    noisy_datasets[key_name] = df_noisy
    print(f"Generato dataset con {percentage}% di rumore ({n_to_flip}
    etichette invertite).")

return noisy_datasets

```

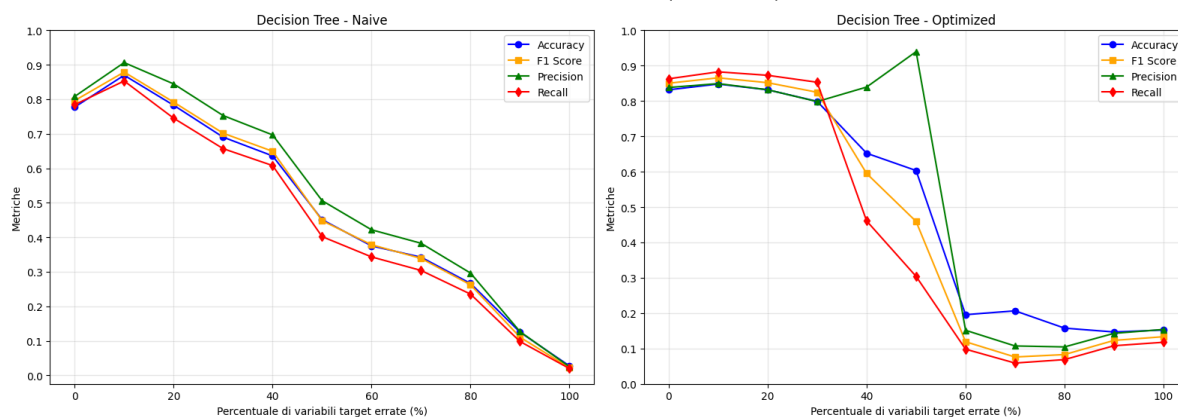
Analisi risultati

Come prima cosa sono andato a visualizzare come evolve la distribuzione della feature target al crescere del rumore.

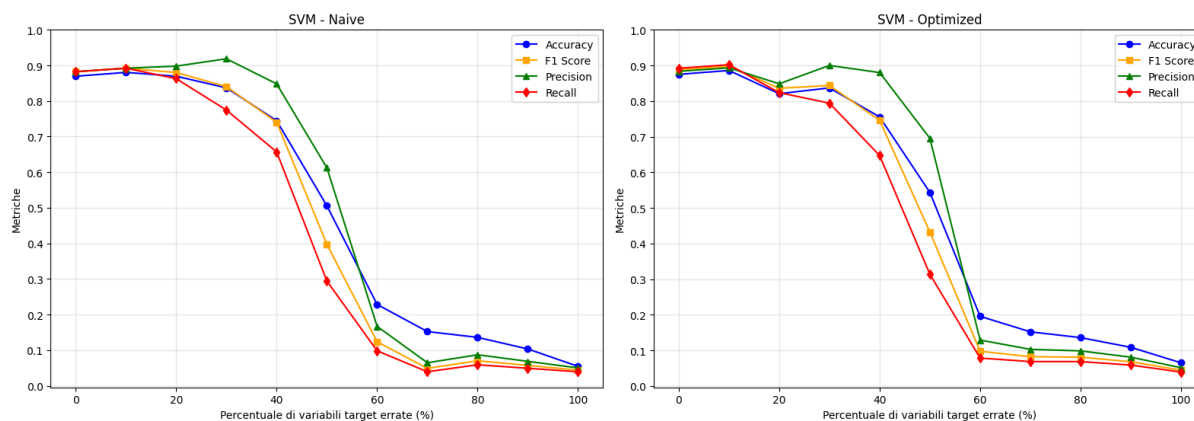


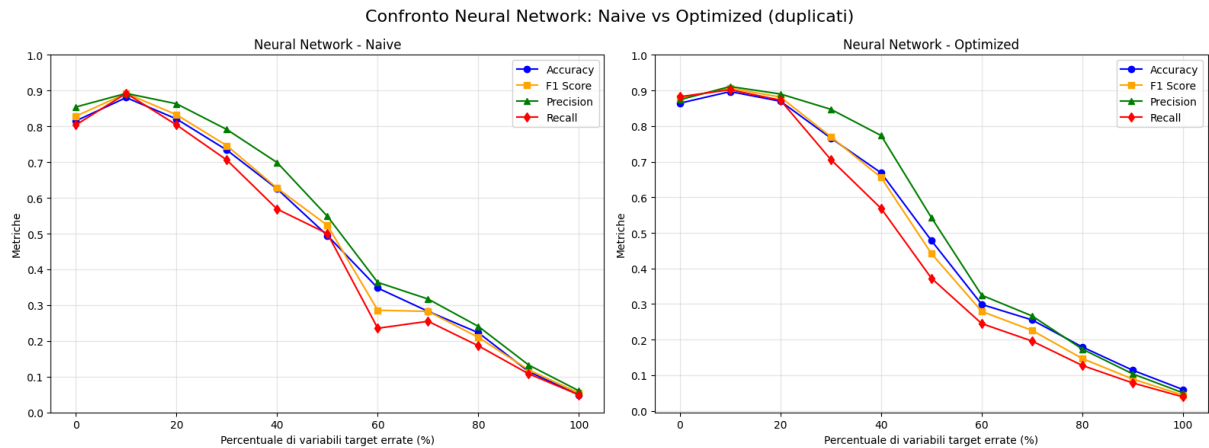
Subito dopo ho creato i grafici che mostrano l'andamento delle metriche al crescere della percentuale di rumore aggiunta alla feature target:

Confronto Decision Tree: Naive vs Optimized (duplicati)



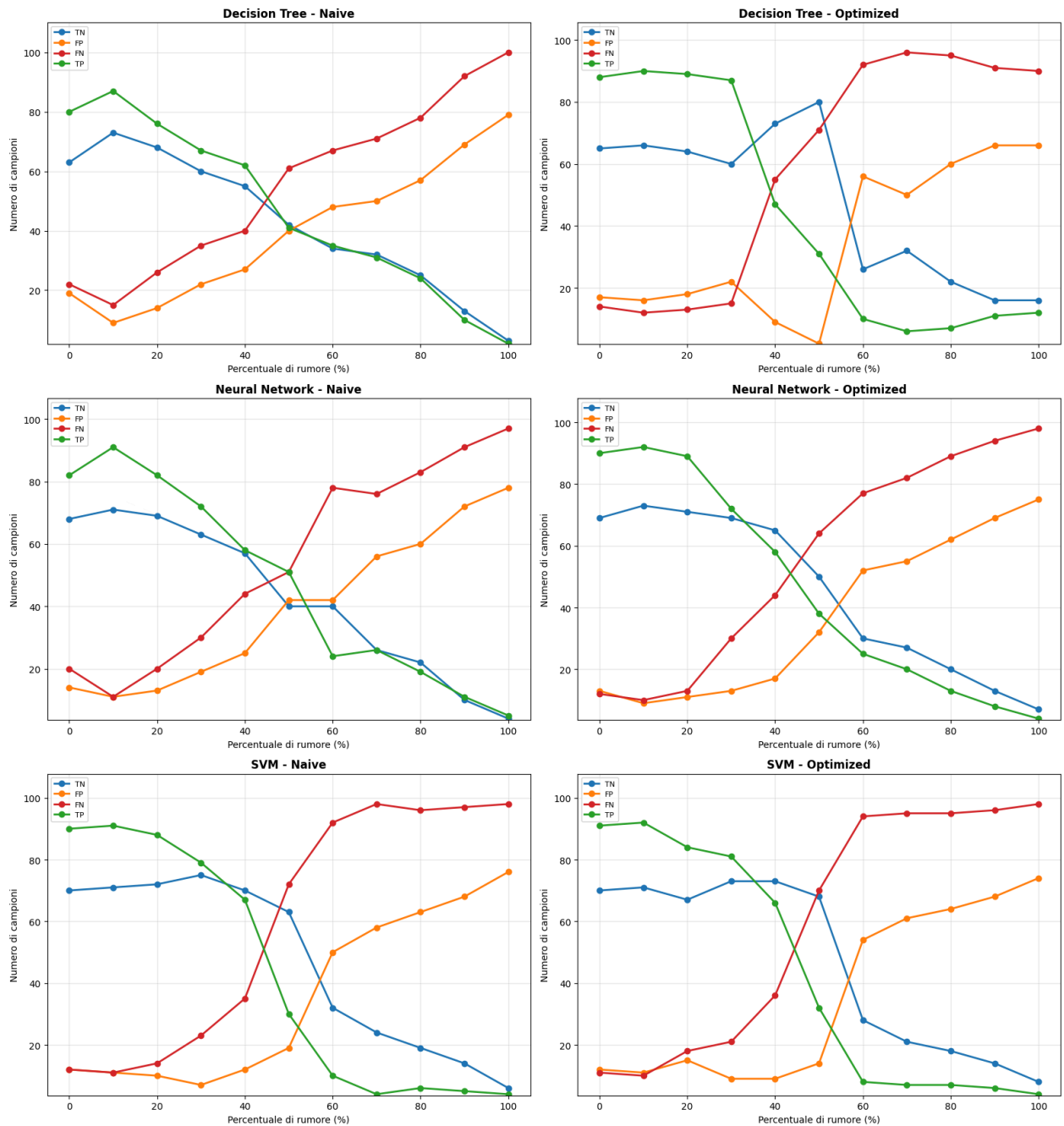
Confronto SVM: Naive vs Optimized (duplicati)





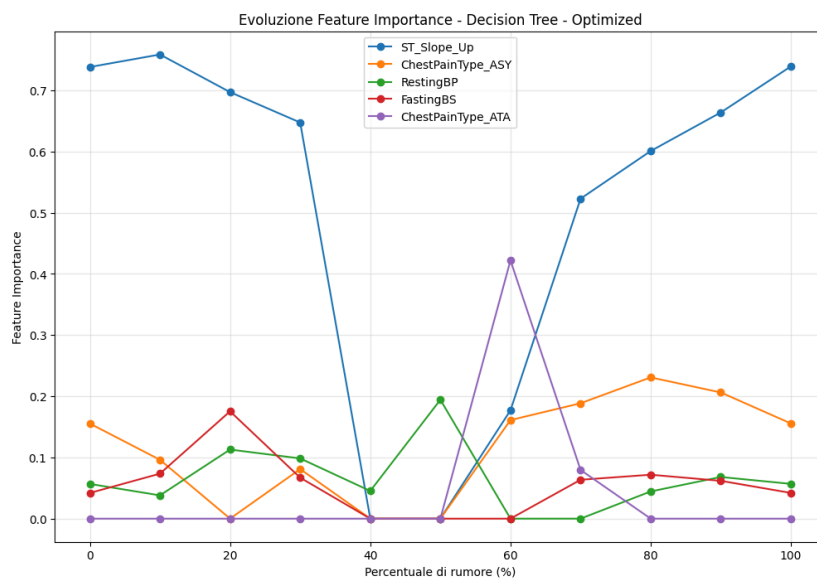
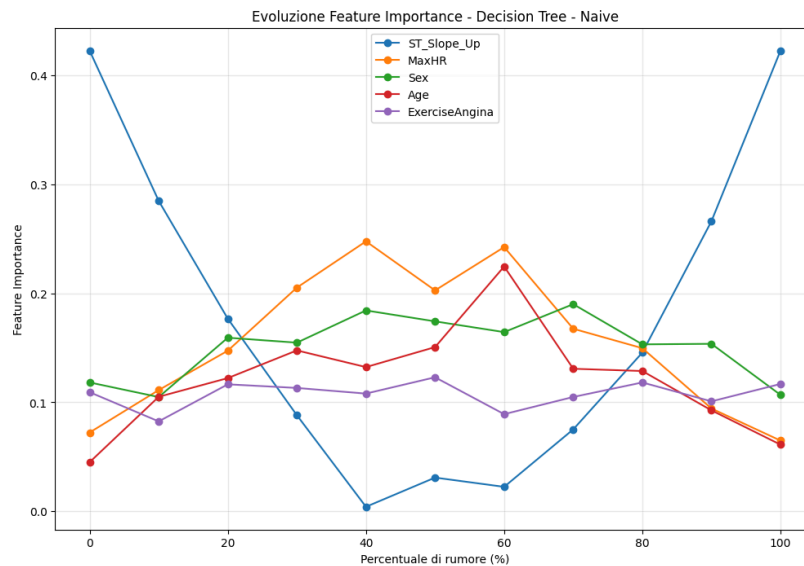
I risultati mostrano un degrado progressivo delle performance per tutti i modelli all'aumentare della percentuale di corruzione della feature target. Le performance dei modelli sembrano resistere fino ad un livello di sporcamento del 20%, oltre al quale si ha una discesa rapida delle performance. L'albero di decisione naive e la rete neurale presentano un decadimento più lineare, mentre l'albero ottimizzato e SVM mostra un andamento non monotono con un picco di errore intorno al 50% di corruzione.

Di seguito invece riporto l'andamento delle matrici di confusione al crescere del livello di degradazione delle feature target.



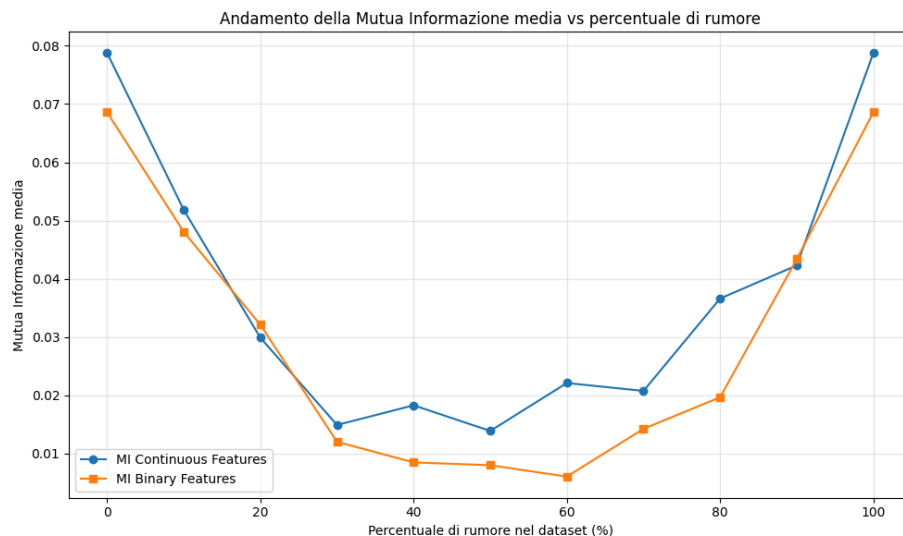
Dai grafici si può osservare, anche se con andamento irregolare, un trend progressivo di decrescita dei True Positive e True Negative al crescere della percentuale di rumore. Al contrario si registra una crescita progressiva dei Falsi Positivi e Falsi negativi al crescere della percentuale di rumore. Questo trend rispecchia in pieno le aspettative teoriche.

Di seguito ho invece riportato l'evoluzione dell'importanza delle feature negli alberi di decisione



Dai grafici risulta come la feature più importante `ST_Slope_Up` crolla di importanza intorno al 50% del livello di contaminazione che nell'albero di decisione ottimizzato corrisponde anche ad un'accelerata del peggioramento delle performance registrate. Infatti appena si torna ad utilizzare `ST_Slope_Up` si ha un trend di leggera crescita delle performance confermando l'importanza di questa feature per questo modello. Lo stesso trend si ha anche nell'albero di decisione naive, dove però non si ha un picco negativo di performance intorno al 50% ma invece si ha una decrescita graduale. Questo può essere motivato da una scelta migliore delle feature sostitutive ad `ST_Slope_Up`.

Infine riporto anche i grafici sulla mutua informazione registrata al crescere del livello di degradazione della feature target.



Il grafico della mutua Informazione media mostra un andamento a U, con valori minimi intorno al 50% di rumore. Questo conferma che la relazione statistica tra features e target si indebolisce progressivamente fino al punto medio di corruzione, per poi rafforzarsi nuovamente man mano che il rumore diventa così consistente da stabilire una nuova relazione invertita. Tale pattern spiega l'andamento osservato nelle performance dei modelli, che tendono a performare peggio quando la relazione features-target è statisticamente ambigua (attorno al 50% di rumore).

Conclusioni

L'analisi condotta sulla robustezza dei modelli di Machine Learning (Decision Tree, SVM, Neural Network) a fronte di diverse tipologie di degradazione del dataset ha permesso di delineare un quadro chiaro delle vulnerabilità e dei punti di forza di ciascun algoritmo.

Dall'analisi di tutti gli esperimenti emerge una gerarchia di robustezza ben definita:

1. **Support Vector Machine (SVM):** Si è confermato il modello più solido. Ha mantenuto performance elevate e stabili nella maggior parte degli scenari, cedendo significativamente solo in presenza di livelli estremi di rumore sul target o sulle feature binarie.
2. **Reti Neurali (Neural Networks):** Hanno mostrato una robustezza paragonabile a SVM. Tuttavia, il costo computazionale (tempi di training e ottimizzazione) è risultato nettamente superiore senza garantire un vantaggio prestazionale che ne giustifichi sempre l'utilizzo rispetto a SVM.
3. **Decision Tree:** È risultato il modello più fragile. In particolare, la versione **Naive** ha sofferto pesantemente di overfitting e di instabilità al crescere del rumore. La versione **ottimizzata** ha mitigato questi problemi, ma ha comunque mostrato un crollo delle performance anticipato rispetto a SVM e Reti Neurali in scenari critici come i valori mancanti.

È stato inoltre confermato che l'**ottimizzazione degli iperparametri** gioca un ruolo cruciale non solo per massimizzare l'accuratezza su dati puliti, ma soprattutto come forma di

"sicurezza" contro i dati sporchi: i modelli ottimizzati hanno mostrato curve di degrado molto più dolci rispetto alle controparti naive.

Classificazione delle Tipologie di Rumore: In base all'impatto osservato sulle metriche (Accuracy, F1-Score, AUC), è possibile classificare le tipologie di rumore dalla più critica alla meno dannosa per questo specifico dataset:

1. **Rumore sul Target (Flip Noise):** Si è rivelata la tipologia di errore più distruttiva. Poiché il modello apprende cercando di mappare le feature su un target, corrompere quest'ultimo distrugge la logica stessa dell'apprendimento, portando le performance a quelle di un classificatore casuale molto più rapidamente rispetto ad altri disturbi.
2. **Rumore sulle Feature Binarie (Flip Noise):** Estremamente dannoso. Dato che il dataset si affida pesantemente a feature binarie discriminanti (come ST_Slope_Up o ExerciseAngina), invertire questi valori compromette la struttura informativa principale del dataset, come evidenziato dall'andamento a U della mutua informazione e dal crollo delle metriche.
3. **Valori Mancanti (Missing Values):** Problematici ma gestibili. I modelli robusti (SVM, NN) e metodi di imputazione avanzati (KNN, Iterative) hanno permesso di mantenere buone performance fino a percentuali molto alte di dati mancanti, rendendo questo problema meno critico dei precedenti se gestito correttamente.
4. **Outliers e Duplicati:** I meno problematici.
 - Gli **Outliers** hanno avuto un impatto limitato poiché il dataset contiene poche feature continue e i modelli sono riusciti a isolarli efficacemente.
 - I **Duplicati** non sono andati ad intaccare le performance dei modelli, che sono rimaste praticamente invariate.

In conclusione, lo studio conferma che l'**SVM ottimizzato** è il modello più performante, dimostrando ottime performance sia sul dataset pulito, sia con le diverse tipologie di rumore introdotte. La sua struttura lo rende particolarmente efficiente e adatto a dataset di dimensioni contenute come quello analizzato. Tuttavia, l'analisi ha evidenziato che anche il modello migliore ha dei limiti intrinseci legati alla qualità del dataset: mentre errori come **valori duplicati** o **outliers** sono stati gestiti efficacemente, il rumore sulle feature target e sulle feature binarie si è rivelato critico, degradando rapidamente le performance. Ciò dimostra che, sebbene la scelta del modello sia fondamentale (con SVM come opzione preferibile), la qualità del dataset rimane il fattore prioritario per ottenere un modello con buone performance.

Codice Sorgente

Ulteriori dettagli e risorse complete relative a questo progetto, sono reperibili nella mia pagina Github: https://github.com/andreaspagnolo/architetture_dati. All'interno, è possibile trovare il codice sorgente integrale (inclusi i notebook ipynb per il preprocessing, le degradazioni del dataset e l'addestramento dei modelli), nonché tutti i grafici generati durante l'analisi, compresi numerosi grafici aggiuntivi non incluse nella relazione per motivi di spazio, come distribuzioni dettagliate per singola feature, matrici di confusione complete e andamenti metriche per ogni step di degradazione.